Taming Multi-Dimensional Skew in Sparse Matrix Multiplication with Contention-Aware Scheduling [Technical Report]

Wentao Huang
National University of Singapore
huangwentao@u.nus.edu

Qiming Huang
National University of Singapore
huang_qiming@u.nus.edu

Yunhong Ji
Renmin University of China
jiyunhong@ruc.edu.cn

Mian Lu
Phancy Group Co., Ltd.
lumian@4paradigm.com

Yuqiang Chen
Phancy Group Co., Ltd.
chenyuqiang@4paradigm.com

Bingsheng He
National University of Singapore
dcsheb@nus.edu.sg

Kian-Lee Tan
National University of Singapore
dcstankl@nus.edu.sg

Abstract

Sparse general matrix-matrix multiplication (SpGEMM) is a key kernel in graph analytics, relational queries, and machine learning. Yet, its performance remains a significant challenge, due to the inherent sparsity and, more critically, the prevalent skew patterns often found in real-world workloads. This paper investigates this problem and demonstrates that an unbalanced workload distribution is the main culprit, harming performance through two co-existing root causes: first, *multi-dimensional skew*, where imbalance across computation cost, output density, and data locality cannot be captured by any single balancing metric; and second, *cache contention*, where dense substructures induce severe on-chip interference.

To that end, we propose adaptive morsel scheduling (AMS), a dynamic scheduling framework that decouples skew dimensions via fixed-result-size morsels and integrates contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact) to alleviate cache-level contention, alongside a hybrid accumulation mechanism (HYB) that adaptively selects the best accumulator per morsel. Evaluations across matrix squaring, relational queries, graph analytics, and other sparse matrix kernels show that AMS significantly outperforms state-of-the-art methods on skewed workloads while maintaining comparable performance on non-skewed ones. These gains, along with improved bandwidth utilization, remain valid across a wide range of workloads and hardware systems.

PVLDB Reference Format:
Wentao Huang, Qiming Huang, Yunhong Ji, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. Taming Multi-Dimensional Skew in Sparse Matrix Multiplication with Contention-Aware Scheduling [Technical Report]. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/fukien/contention-aware-spgemm.

1 Introduction

Matrix multiplication is a fundamental computational kernel, and its sparse variant, sparse general matrix-matrix multiplication (SpGEMM), is especially important in practice. The reason is that sparse matrices are a natural model for real-world graph data, which makes SpGEMM a core building block for applications such as graph analytics, entity resolution, and machine learning [38, 61, 76, 79, 87, 103, 126]. Recently, SpGEMM has emerged as a transformative method for processing relational join-aggregate and join-project queries [31, 51–53, 58, 67, 100–102]. By treating relational tables as algebraic sparse matrices, entire query pipelines can be executed as unified algebraic operations. This enables aggregation or projection to be performed *on-the-fly* alongside the join, effectively bypassing the prohibitive costs of materializing excessive intermediate results (see Figure 1 for a comparison) . In all such cases, the performance of the application is dictated by the efficiency of the underlying parallel SpGEMM execution. Reflecting its critical role, numerous studies have investigated SpGEMM optimization, reaching a broad consensus that it is a fundamentally bandwidth-intensive kernel where throughput is ultimately governed by available memory bandwidth [45, 84].

Though it is bandwidth-bound, running SpGEMM on a high-bandwidth platform does not necessarily translate to higher throughput [1] . This is primarily attributed to the *load imbalance* caused by the widely observed skew patterns in real-world workloads [13, 27, 66, 72, 94, 112, 125]. For instance, in a typical OLAP ClickHouse benchmark [96], the top 5% of users generate 24.4% (47.8 million) of all clicks, while the median number of clicks per user is only 2. Similarly, in a common PageRank workload [53, 118], the top 1.8% of pages receive more incoming links than the remaining 98.2% combined. From a matrix perspective, such skewed key frequencies and power-law graph degrees translate into highly irregular nonzero structures, resulting in significant load imbalance. When running

[1]While SpGEMM above a certain density threshold can be compute-limited, the growing compute-bandwidth gap [41, 74, 91, 108] makes compute progressively less likely than bandwidth to be the primary bottleneck.

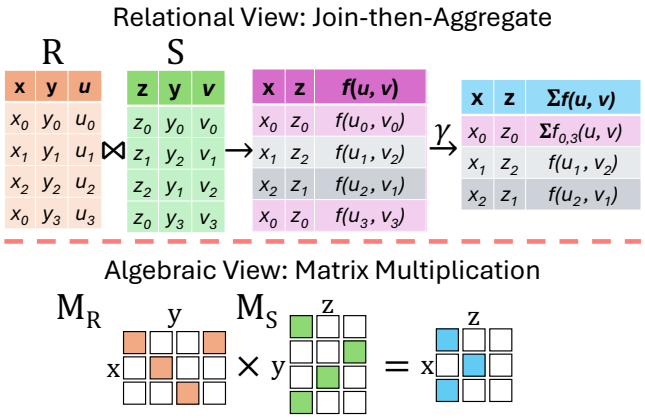


Figure 1: Comparing two approaches for evaluating a join-aggregate query, " *SELECT x, z, SUM(f(u,v)) FROM R, S WHERE R.y = S.y GROUP BY x, z* ", where f denotes a mathematical function. The "relational view" (top) joins tables $R(x, y, u)$ and $S(z, y, v)$ on attribute y followed by an aggregation, a process that produces excessive intermediate results, such as $(x_0, z_0, f(u_0, v_0))$ and $(x_0, z_0, f(u_3, v_3))$, which are ultimately aggregated into one final group-by result (shaded in light pink). The "algebraic view" (bottom) models the operation as a matrix product $M_R \times M_S$ where y serves as the shared dimension to compute the aggregate result directly, bypassing the materialization of costly intermediate results.

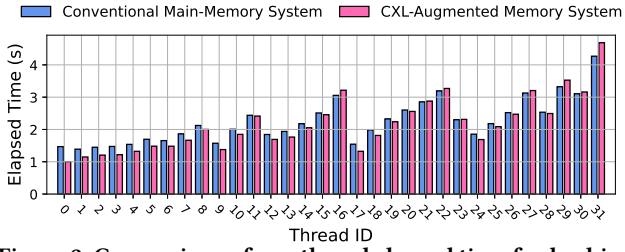


Figure 2: Comparison of per-thread elapsed time for hashing-based SpGEMM [84] on a G500 skewed matrix (scale=17, *edgefactor*=16) [22] between a conventional main-memory system (28.50 GB/s bandwidth) and a CXL-augmented system (46.50 GB/s bandwidth) (see § 4 for experimental setups).

SpGEMM on these workloads, a small subset of cores becomes overloaded while others remain underutilized [38, 45, 84]. This not only limits compute efficiency but also prevents full utilization of the available memory bandwidth, leaving system resources idle, and therefore, rendering high-bandwidth platforms less effective.

Moreover, the challenge of addressing load imbalance in skewed workloads is particularly critical and is further exacerbated by the "memory scaling wall," where memory technology advancements lag behind those of processing units [91, 108]. This trend results in decreasing memory bandwidth and capacity per processing core [57, 116], which further compounds the load imbalance problem for two reasons:

First, decreasing bandwidth-per-core has an acutely negative impact on SpGEMM workloads with load imbalance, where a few straggling threads drag down performance. Since these threads largely rely on core-level bandwidth, their overall performance is inevitably reduced due to the insufficient bandwidth-per-core caused by the "memory scaling wall."

Second, the prevalent approach to compensating for decreasing bandwidth-per-core, namely the adoption of heterogeneous memory expansion technologies like CXL [56, 71, 116, 117], can inadvertently amplify this problem. Since CXL-attached memory typically exhibits higher access latency than local DRAM [2] , straggling threads that run disproportionately long are more exposed to CXL's latency penalty, further widening the gap between fast and slow threads. Figure 2 illustrates this using a hashing-based SpGEMM algorithm [84] on a skewed workload, where per-thread runtimes vary due to load imbalance. [3] Because total runtime is governed by the slowest thread, the disproportionate latency on stragglers negates the aggregate bandwidth gain, yielding worse overall performance than pure DRAM. This counterintuitive result is also reported in existing CXL studies [77, 78, 104, 107]. As a result, addressing load imbalance is becoming more imperative than ever.

However, addressing load imbalance in SpGEMM is not trivial. While numerous strategies have been proposed [38, 45, 88], we maintain that existing approaches fall short due to two co-existing root causes: *multi-dimensional skew* and *cache contention*, both of which not only hinder load balancing but also degrade overall performance.

First, existing methods fail to accommodate the *multi-dimensional skew* inherent in sparse matrices. Current methods rely on static schemes that balance along a single metric (e.g., flop count) to minimize scheduling overhead. However, skew manifests across multiple dimensions, including computation cost, materialization overhead, and data locality, making it impossible for any single static metric to achieve balanced execution across all threads.

Second, in skewed workloads, existing approaches suffer from significant cache contention, induced by dense substructures that are co-scheduled together. In SpGEMM, irregular sparsity forces practitioners to cache row-wise intermediates to reduce random access costs. However, dense substructures within skewed matrices incur excessive irregular random entry access, which creates heavy cache pressure and even induces significant cache contention, which has long been overlooked as a key performance factor. This cache contention is particularly pronounced under simultaneous multithreading (SMT), which aims to increase computation parallelism but worsens the interference.

In this paper, we address these challenges through a novel approach, *adaptive morsel scheduling (AMS)*. In contrast to de facto methods, AMS revisits *dynamic scheduling*, a strategy conducive to load balancing that is often dismissed due to concerns regarding scheduling overhead and poor performance [45, 84]. Through analysis (§ 2.3), we find that task granularity is the primary performance

[2]Since DRAM is the primary technology used in memory manufacturing, we use the terms "DRAM" and "memory" interchangeably throughout this paper.

[3]We use the following notation for R-MAT graphs [22]: **ER**, **SSCA**, and **G500** denote non-skewed Erdős–Rényi, skewed SSCA, and highly skewed Graph500 graphs, respectively; *scale* sets the number of rows to 2^{scale}, and *edgefactor* sets the average nonzeros per row.

factor: a granularity that is too fine fails to preserve spatial locality and therefore suffers from excessive cache thrashing. Hence, we employ a morsel-wise dynamic scheduling strategy, where a "morsel"[4] is defined as a basic scheduling unit of several consecutive matrix rows. AMS ensures a morsel is large enough to maintain spatial locality, thereby greatly mitigating cache thrashing, and more importantly achieves load balance on skewed workloads through dynamic scheduling, effectively preventing computational idling and improving memory bandwidth utilization.

To alleviate cache contention, AMS introduces two lightweight and generalizable techniques: *contention-aware load balancing (CALB)*, which mitigates cache interference by adaptively scheduling dense morsels, and *data-conscious simultaneous multithreading activator (SMTact)*, which selectively enables SMT based on workload contention. These optimizations are simple in design, yet effective in practice, and easy to generalize to other sparse matrix kernels (e.g., SpMM and SpMV). Importantly, their performance benefits remain effective across heterogeneous bandwidth-boosted systems (e.g., CXL systems), ensuring that latency heterogeneity does not cause performance degradation.

Furthermore, the morsel-wise scheduling strategy enables a *hybrid accumulation mechanism (HYB)* for SpGEMM, where different accumulation algorithms can be selectively applied to morsels based on their intrinsic characteristics, thereby achieving more efficient execution and enhanced overall SpGEMM performance.

The contributions of this paper are summarized as follows:

(1) We examine and characterize two co-existing root causes of performance degradation in skewed SpGEMM workloads: *multi-dimensional skew*, where no single static metric can balance all threads, and *cache contention*, where dense substructures induce severe on-chip interference that existing methods overlook.
(2) We develop a novel morsel-wise dynamic scheduling framework, adaptive morsel scheduling (AMS), that systematically addresses both *multi-dimensional skew* and *cache contention* in SpGEMM. AMS decouples skew dimensions through fixed-result-size morsels, integrates two key techniques, contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact), to alleviate on-chip cache interference, and enables hybrid accumulation mechanism (HYB) for morsel-wise accumulation adaptively. Our approach is straightforward in design, yet novel and generalizable, and effective in practice.
(3) We evaluate AMS across a wide range of workloads, including matrix squaring, relational join-aggregate and join-project queries, graph analytics, and other sparse kernels, on both a main-memory and a CXL-augmented system, and show that it performs on par with state-of-the-art methods on non-skewed workloads while significantly outperforming them on skewed ones, addressing both computational idling and bandwidth underuse.

The rest of the paper is organized as follows. We first present the background and motivation in § 2, followed by a description of AMS in § 3. We then show the experimental results in § 4 and discuss related work in § 5. Finally, we conclude the paper in § 6.

[4]We adopt the term from Leis et al. [73], where it denotes a fixed-size fragment of input data, whereas in our work it denotes a group of consecutive rows with a roughly fixed output size.

Algorithm 1 Parallel Gustavson's Algorithm for SpGEMM

```
1:  Input: CSR matrices A ∈ ℝ^{m×k} and B ∈ ℝ^{k×n}
2:  Output: CSR matrix C ∈ ℝ^{m×n}
3:  parallel for i ← 0 to m − 1 do          ▷ Parallel over rows of A
4:      Initialize empty accumulator for row i
5:      for all (k, A_ik) ∈ row i of A do
6:          for all (j, B_kj) ∈ row k of B do
7:              if j ∈ accumulator then
8:                  accumulator[j] += A_ik · B_kj
9:              else
10:                 accumulator[j] ← A_ik · B_kj
11:             end if
12:         end for
13:     end for
14:     Store accumulator as row i of C
15: end parallel for
```

2 Background and Motivation

We first review state-of-the-art SpGEMM algorithms, then analyze how multi-dimensional skew and cache contention degrade performance under skewed workloads, and finally revisit dynamic scheduling as a promising solution, motivating our AMS approach.

2.1 The SpGEMM Kernel

SpGEMM computes the product $C = AB$, where A, B, and C are all sparse matrices [5] . This form has proven effective across diverse large-scale workloads, including scientific computing, graph analytics, and more recently relational query processing [51, 83, 84, 101]. To reduce memory footprint at scale, sparse matrix formats are widely used, with compressed sparse row (CSR) being the most common representation in existing works [11, 20, 43, 60, 93]. However, due to the irregular and input-dependent sparsity patterns of sparse matrices, predicting the number and locations of nonzeros in the output matrix is often intricate, making memory preallocation challenging. Combined with irregular memory access patterns, these challenges make SpGEMM a fundamentally memory-intensive workload, with performance critically dependent on sustained memory bandwidth across cores.

To address these challenges, SpGEMM is typically divided into two phases: a symbolic phase, which determines the nonzero structure of the output, and a numeric phase, which performs the actual computation based on that structure. In addition, the irregular nature of sparsity makes it difficult to efficiently gather and accumulate scattered contributions to each output entry. To tackle this, prior work has introduced a variety of accumulator designs to aggregate partial results during computation, including sorted variants such as heaps [83, 84], SPA [42, 48], and expand-sort-compress (ESC) [12, 29], as well as non-sorted variants based on hash tables [50, 83, 84].

[5]Throughout this paper, we refer to A and B in $C = AB$ as the left-hand side and right-hand side matrices, respectively.

Building on these data representations, SpGEMM algorithms are commonly implemented using one of three formulations: inner-product [90], outer-product [2, 45], or row-row paradigm (Gustavson's algorithm) [48]. Among these, Gustavson's algorithm operates in a row-wise manner, allowing each row of the left-hand side matrix (A) to be computed independently, enabling straightforward thread-level parallelization without the need for inter-thread synchronization (see Algorithm 1). It is this high degree of parallelism, combined with its efficient memory access patterns, that has made Gustavson's algorithm a dominant formulation for SpGEMM across a wide range of computing platforms.

Following Gustavson's algorithm, several load-balancing techniques have been proposed [2, 4, 9, 18, 24, 110], with static scheduling based on a flop-count scheme emerging as the dominant strategy [83, 84]. This method partitions the rows of matrix A into contiguous regions, each assigned approximately the same number of floating-point operations (i.e., additions plus multiplications). To achieve this, a lightweight preprocessing pass scans matrices A and B to estimate the flop count for each output row of C without performing actual calculations. Based on these estimates, rows are grouped into partitions such that each thread receives a workload with a nearly equal number of floating-point operations. This approach enables static, flop-count-balanced scheduling across threads while avoiding runtime synchronization.

2.2 The Flop-Count-Based Scheme Under Skew

Despite its simplicity and widespread adoption [20, 84], the flop-count-based scheme is fundamentally limited and fails to achieve effective workload balance under skewed patterns. We identify two root causes, an inherent ignoring of multi-dimensional skew and the performance penalty of overlooking cache contention, both of which hurt load balancing and degrade bandwidth utilization.

(1) Ignoring multi-dimensional skew. Skewed SpGEMM workloads can exhibit imbalance across multiple dimensions, such as submatrix density and row-wise nonzero (*nnz*) distribution. However, the flop-count-based scheme equalizes only the total number of floating-point operations, ignoring other forms of skew. As a result, one thread may receive a few dense rows (with high *nnz*), while another is assigned many sparse rows. Although these assignments may have similar total flop counts, their structural differences lead to divergent runtime behaviors. Dense rows, in particular, induce "clustering" [6] effects during processing, which manifest as temporal bursts of memory and cache pressure. These effects significantly degrade performance, and the extent of impact varies by accumulation strategy [29, 42, 83]. Therefore, the scheme is inherently unable to account for these skew dimensions.

Additionally, thread-level partitions often exhibit different compression factors (cf) [7] , which leads to divergent trends in computation and materialization times, making it challenging to effectively balance these two workload components. Consider Figure 3: using Gustavson's algorithm, partition-0 (the upper two rows of A) requires 12 flop counts [8] , producing 6 nonzeros; partition-1 (the bottom three rows) requires 11 flop counts but produces 9 nonzeros. Despite both involving comparable amounts of computation, the second has a larger output, inducing more materialization cost.

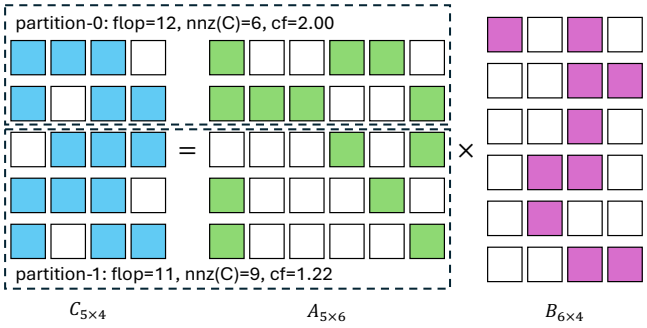


Figure 3: An illustration of $C = AB$ in SpGEMM (colored cells denote nonzero elements), where the two dashed boxes represent partitions with comparable flop counts but differing numbers of output nonzeros and compression factors.

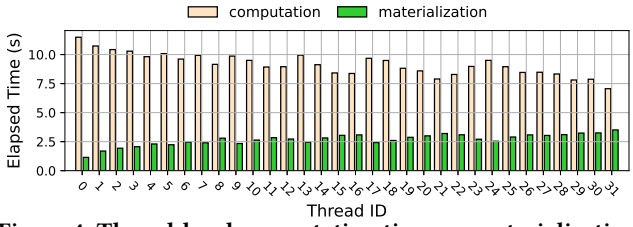


Figure 4: Thread-level computation time vs. materialization time for hashing-based SpGEMM on a G500 skewed matrix (scale=19, *edgefactor*=8).

Empirical data on a skewed workload confirms this disparity, as Figure 4 shows: materialization time varies across threads, with computation time also exhibiting thread-level variation, but in the opposite trend. Moreover, these complexities are further compounded by hardware variability: performance sensitivity to the relative costs of computation versus memory access differs considerably across hardware platforms. This architectural variance makes it exceedingly difficult for purely flop-count-based schemes to establish a load balance that is both universally effective and readily generalizable across diverse machines. Thus, such schemes are inherently limited in delivering robust load balancing.

(2) Overlooking cache contention. Existing methods also overlook the performance impact of cache contention during parallel execution. In Gustavson's algorithm, each thread maintains an exclusive row-level accumulator whose size scales with the nonzero density of its assigned rows. Reading Figure 3 row-wise, each row of A gathers the B rows indexed by its nonzeros into that accumulator, so cf measures the nonzeros of B streamed per accumulator entry; a high-cf row thus places significantly greater pressure on shared caches than a low-cf row. As high-cf partitions generally contain more high-cf rows and Gustavson's algorithm is typically parallelized row-wise, they are more prone to inter-thread cache interference than low-cf partitions. Figure 5(a) shows this effect using the same skewed matrix as in Figure 4. When all threads compute their assigned default partitions, the runtime of a high-cf partition

[6] We use the term "clustering" to describe this effect, following the notion of "primary clustering" in open-addressing hashing.

[7] The compression factor (cf) is defined as the number of flop counts divided by the nnz of the result matrix C, i.e., $cf = \frac{flop}{nnz(C)}$ [45].

[8] Only multiplications are counted, as the number of additions is identical.

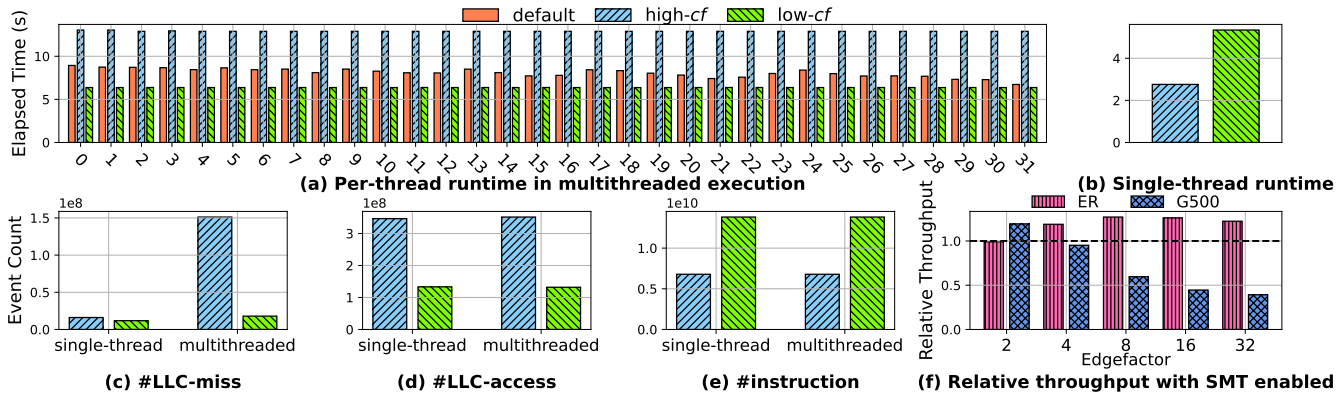


Figure 5: (a) Per-thread computation time of HASH-based SpGEMM on a G500 skewed matrix (scale=19, *edgefactor*=8) under three settings: all threads execute their default partitions; all execute a high-cf partition (thread-0's); all execute a low-cf partition (thread-31's). (b) Computation time of the low-cf and high-cf partitions under single-threaded execution. (c)–(e) LLC misses, LLC accesses, and instruction counts for the low-cf and high-cf partitions under both single-threaded and multithreaded execution. (f) Normalized throughput of HASH-based SpGEMM with SMT enabled on ER and G500 matrices (scale=18) across varying *edgefactor* values. The black dashed line marks the relative throughput without SMT.

(originally assigned to thread-0 in the flop-count-based scheme) increases substantially: from 2.76 seconds in single-threaded execution (see Figure 5(b)) to 8.93 seconds under default multithreaded partitioning, and further to 13.01 seconds if all threads execute this high-cf partition. Note that no sharing benefit arises in this case, as each thread operates on its own exclusive accumulator; instead, these accumulators compete for the same shared cache, amplifying interference as more threads process high-cf regions. In contrast, a low-cf partition (originally assigned to thread-31) sees only a modest runtime increase, from 5.36 to 6.73 seconds.

Further profiling reveals that cache contention is the root cause of this disparity: Figure 5(c)–(e) shows a sharp rise in last-level cache (LLC) misses for the high-cf region, while instruction count and LLC accesses remain relatively stable, confirming cache contention as the primary bottleneck [9] .

The problem is further exacerbated under simultaneous multithreading (SMT): Figure 5(f) shows that non-skewed workloads (e.g., ER matrices with typically low cf) benefit from SMT, whereas skewed workloads (e.g., G500 matrices with high-cf subregions) experience increasing throughput degradation as density grows, since higher density amplifies the skew and thus intensifies SMT-induced contention. This underscores the importance of contention-aware scheduling; neglecting this factor not only compromises load balance but also degrades overall SpGEMM efficiency.

Ignoring multi-dimensional skew and overlooking cache contention together compound into a macro-level penalty on system-wide bandwidth utilization. As Figure 6 illustrates on a skewed workload, bandwidth resources are only fully utilized during the initial burst of parallel execution; as straggler cores emerge, system-wide bandwidth consumption starts to drop, leaving expensive memory resources idle. This renders prevailing bandwidth expansion technologies, such as CXL, far less effective, as the slowest thread governs total runtime, and its prolonged latency [104] further widens the gap despite the increased aggregate bandwidth

[9]Write-out time is excluded to directly address computation-level contention.

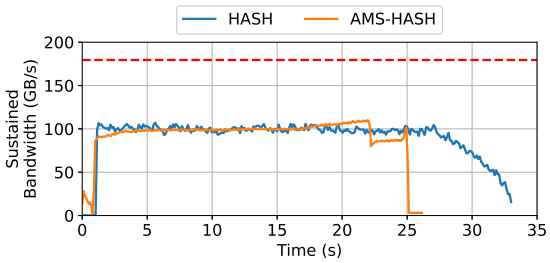
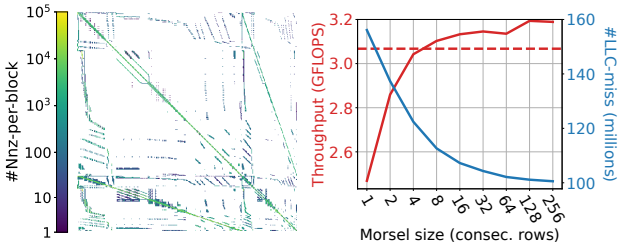


Figure 6: Comparison of sustained bandwidth of the numeric phase between flop-count-based hashing-based SpGEMM and the AMS variant on a G500 skewed workload (scale=19, *edgefactor*=16); the horizontal red dashed line denotes system memory bandwidth (179.50 GB/s).

available. In contrast, a contention-aware load-balancing approach (our AMS method) sustains markedly higher bandwidth execution.

2.3 Revisiting Dynamic Scheduling

Having established that both multi-dimensional skew and cache contention undermine existing scheduling strategies, we now revisit the scheduling paradigm that enables our solution. Specifically, we revisit *dynamic scheduling* [84, 88]: rows of A are placed into a shared task pool, and threads repeatedly fetch and process one row at a time, achieving nearly 100% load balance. While simple and theoretically well-balanced, this approach has been shown to underperform compared to flop-count-based scheduling and has largely been dismissed in recent literature, which attributes the performance penalty primarily to the synchronization overhead associated with dynamic task dispatch [45, 84]. However, we contend that it is task granularity, rather than synchronization overhead, that plays the dominant role in determining performance. When task size is carefully managed, dynamic scheduling can match, if not outperform, flop-count-based schemes.



(a) NNZ-pattern (b) Throughput & LLC-miss

Figure 7: (a) Nonzero pattern of the `vas_stokes_1M` matrix. (b) Throughput and LLC misses of hashing-based SpGEMM on `vas_stokes_1M` using dynamic scheduling with varying morsel sizes (measured in consecutive rows per morsel); the horizontal red dashed line indicates the throughput of the flop-count-based scheme.

Take a real-world SpGEMM workload, `vas_stokes_1M` [30, 98], for example. The matrix exhibits clear spatial locality across rows, particularly within blocks of consecutive rows (Figure 7(a)). We benchmark the performance of hashing-based SpGEMM using dynamic scheduling with varying task granularities (i.e., the number of consecutive rows per task). Figure 7(b) shows that throughput improves with increasing granularity, plateauing around 128 rows, and surpasses its flop-count-based counterpart due to better load balance. Further analysis of LLC miss counts reveals that larger task sizes result in fewer misses, suggesting better preservation of spatial locality. It is worth noting that many real-world matrices exhibit strong spatial locality in their nonzero patterns, and prior works have introduced several offline reordering algorithms to enhance locality for downstream operations [4, 24, 110]. Based on these observations, we maintain that dynamic scheduling remains a practical and efficient strategy as long as spatial locality is effectively preserved. We therefore build on this insight for the design of our AMS approach.

3 Methodology

This section presents our proposed framework, adaptive morsel scheduling (AMS). We begin with the core concept of morsel-wise dynamic scheduling, our strategy for addressing multi-dimensional skew and achieving efficient load balancing in SpGEMM. Building upon this foundation, we present two key techniques, contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact), designed specifically to alleviate on-chip cache contention. Furthermore, we explain how the morsel-wise framework enables our hybrid accumulation mechanism (HYB) approach for optimizing SpGEMM execution. Finally, we discuss three broader implications of AMS, namely the interplay between load balance and cache efficiency that governs the benefit of bandwidth-expanded memory, the carry-over of the underlying design principle to relational and graph processing, and the extension of AMS to other sparse matrix multiplication kernels.

We note that AMS targets the numeric phase of SpGEMM, which dominates both computation and memory traffic. While the same ideas can be applied to the symbolic phase, the benefits are less pronounced; for simplicity, we use standard dynamic scheduling with fixed-size morsels for that phase.

Before detailing our techniques, we briefly digress to outline three key design principles that an efficient SpGEMM solution should satisfy, all of which are met by our proposed AMS.

(1) **Simple and lightweight.** SpGEMM has been extensively optimized over decades [38], and any added complexity can degrade its performance. This is especially critical when SpGEMM is repeatedly invoked as a core kernel in real-world systems [20, 46, 60, 76, 84, 111]. Thus, any proposed method must be simple and lightweight to avoid introducing significant execution penalties.
(2) **Non-disruptive to non-skewed workloads.** While skewed patterns are common in real-world workloads, non-skewed distributions (e.g., uniform distribution, symmetric distribution) also arise frequently. A design that benefits skewed cases but penalizes non-skewed ones forces practitioners to make case-dependent trade-offs, increasing system complexity and operational overhead.
(3) **Generalizable across algorithms and platforms.** An effective SpGEMM solution should generalize across algorithmic variants and hardware platforms. In practice, SpGEMM is used in diverse contexts with varying requirements, e.g., numerical computing often demands sorted output [29], while relational and graph queries typically accept non-sorted results [4, 24, 43]. Also, it should adapt to different hardware architectures, where sensitivities to compute and memory bottlenecks vary. A practical solution must therefore accommodate both, supporting sorted and non-sorted pipelines while maintaining robust performance across platforms.

3.1 Morsel-Wise Dynamic Scheduling

We previously demonstrated that, by using morsels composed of a sufficient number of consecutive rows, dynamic scheduling is able to effectively preserve spatial locality in sparse matrices. This method significantly reduces LLC misses with negligible synchronization overhead (§ 2.3). We therefore adopt this line of design for SpGEMM execution.

While scheduling work in morsels of a fixed row count helps reduce cache thrashing, we argue that this approach leaves room for further improvement, as it does not address the *multi-dimensional skew* identified in § 2.2. Specifically, it has two limitations: (1) Fixed-row-count morsels do not account for differences in flop count or result *nnz*, making it difficult to balance computation and materialization costs. This limitation is particularly pronounced in skewed workloads, where high-cf submatrices incur high computational expense while others involve substantial write-out overhead. Due to such disparities in two dimensions, modeling inter-thread cache contention becomes challenging. (2) This fixed-row strategy also fails to control the size of the resulting output. A single morsel containing a few dense rows, particularly in skewed workloads, can produce more data than the CPU cache can accommodate, making it challenging to construct a cache-aware analysis model [50, 82, 95].

To address these issues, we propose scheduling morsels based on a fixed result size, i.e., a target $nnz(C)$ per morsel. Recall that SpGEMM consists of two phases, where the symbolic phase determines the per-row nonzero structure. We thus leverage this information to form morsels of consecutive rows in an adaptive manner, such that each morsel contains approximately the same

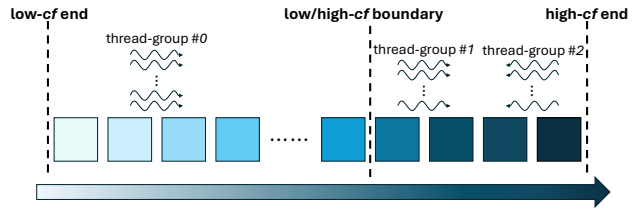


Figure 8: Thread scheduling with contention-aware load balancing (CALB). Darker colors indicate higher compression factors (cf).

number of output nonzeros. By fixing the materialization cost per morsel, this approach normalizes one dimension of skew, allowing us to focus on the remaining computational cost dimension. It also decouples cf from the output size that flop-count-based partitioning leaves uncontrolled: with $nnz(C)$ fixed, cf varies solely with the flop count. Additionally, to facilitate on-chip cache modeling, we limit the morsel size to half of the L2 cache capacity [10].

This strategy is simple and efficient, and it forms the foundation for our subsequent designs. More importantly, it introduces no additional algorithmic phase, fully aligning with our design principle of being **simple and lightweight**.

3.2 Contention-Aware Load Balancing

We now elaborate on our alleviation of inter-thread on-chip cache contention, which can degrade SpGEMM performance even under balanced workload distribution. Since we form morsels based on a fixed result size, we can easily compute the compression factor (cf) for each morsel. Recall from § 2.2 that contention originates at the row level: a high-cf row streams more nonzeros of B per accumulator entry (see Figure 3), and is thus contention-prone regardless of how rows are grouped. With $nnz(C)$ fixed per morsel, a high-cf morsel likewise carries more computation against an identical footprint, and remains more prone to contention than a low-cf one. Therefore, we aim to co-execute high-cf and low-cf morsels concurrently, alleviating cache contention and reducing susceptibility to inter-thread interference.

To that end, we first sort all morsels in ascending order of cf and empirically set a boundary between low-cf and high-cf regions, typically between the 80th and 95th percentile. This is motivated by the observation that most skewed workloads follow a power-law distribution [4, 24, 110, 125], where a small fraction of dense rows contribute to the majority of the workload. We then divide the available threads into three groups: thread group #0 starts from the low-cf end and progresses toward the high-cf end; group #1 begins at the low/high-cf boundary and proceeds toward the high-cf end; group #2 starts at the high-cf end and moves backward toward the boundary (see Figure 8). The majority of threads are allocated to group #0 to handle low-cf tasks, while smaller subsets are assigned to groups #1 and #2 for high-cf regions. This arrangement ensures that only a limited number of high-cf morsels are processed concurrently. Dividing the remaining threads into groups #1 and #2 further helps mitigate cache contention by avoiding simultaneous execution of multiple contention-prone morsels at the high-cf end. Meanwhile, co-running low-cf morsels, which are less sensitive to cache interference, alongside high-cf morsels helps absorb contention effects and improves overall execution efficiency.

While this scheme alleviates cache contention, it does not guarantee load balance. Some thread groups may finish early and become idle, leading to underutilization of computational and memory bandwidth resources. To address this, we allow early-finished threads to assist other active groups. Specifically, if group #0 reaches the boundary early, it merges with group #1 to continue processing toward the high-cf end. Likewise, early-finished threads from groups #1 and #2 join group #0 to help complete tasks in the low-cf region. This cooperative mechanism keeps all threads active, ensuring efficient use of computation and memory bandwidth.

The CALB scheme is conceptually simple, yet novel and highly effective. To the best of our knowledge, it is the first SpGEMM load-balancing technique that explicitly considers on-chip cache contention as a scheduling factor and thereby delivers tangible performance benefits. Moreover, the method is platform-agnostic and can be seamlessly integrated into any SpGEMM algorithm based on Gustavson's formulation, adhering to the design principle of being **generalizable across algorithms and platforms**.

3.3 Data-Conscious Simultaneous Multithreading Activator

While SpGEMM is generally considered bandwidth-bound, its computational phase still incurs a non-negligible overhead. This overhead can typically be mitigated by leveraging simultaneous multithreading (SMT), which increases the logical core count to effectively improve computational throughput. However, as shown in Figure 5(f), this performance benefit from SMT is primarily observed in non-skewed workloads; in contrast, skewed workloads degrade significantly when SMT is enabled.

We attribute this disparity to the increased on-chip cache pressure from SMT. Although SMT increases the logical thread count, it does not expand the physical cache. Consequently, high-cf morsels suffer from intensified cache contention, an issue that is particularly exacerbated in skewed workloads where cf values are disproportionately large for a few morsels. These specific morsels are highly vulnerable to SMT-induced interference, which in turn leads to substantial performance degradation.

To mitigate this issue while preserving the computational benefits of SMT for non-skewed workloads, we adopt a selective activation strategy of SMT, namely data-conscious simultaneous multithreading activator (SMTact). Specifically, we set a threshold based on the highest cf observed. SMT is disabled if the workload contains any morsels exceeding this threshold, and enabled otherwise. This strategy ensures that SMT is applied only when the workload is less prone to cache contention [11]. The approach is simple yet effective, as it balances the computational benefits of SMT against the risk of elevated cache contention, aligning with our second design principle of being **non-disruptive to non-skewed workloads**.

[10] We empirically find that morsel sizes ranging from $\frac{1}{8}$ to $\frac{1}{2}$ of the L2 cache capacity offer the best trade-off between performance and cache contention.

[11] We also experiment with using quantiles of the cf distribution instead of the maximum, and observe comparable performance with an appropriately chosen threshold.

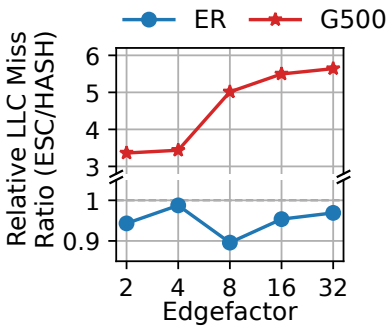


Figure 9: The LLC miss ratio of ESC normalized to HASH (i.e., LLC Misses of ESC / LLC Misses of HASH), evaluated on ER and G500 datasets at Scale 18 with varying edgefactors (higher edgefactors suggest denser workloads).

3.4 Hybrid Accumulation Mechanism

Due to the inherently sparse and irregular nature of SpGEMM, computing a single output row generates multiple intermediate sparse products. These products must then be merged, a process that requires appropriate accumulators to mitigate two challenges: severe irregular memory access patterns and the high cost of managing collisions for identical column indices. Recognizing these challenges, numerous works have focused on developing efficient accumulators [29, 42, 84]. However, the effectiveness of any given accumulation strategy varies dramatically based on the characteristics of the underlying matrix morsel. This makes a static, one-size-fits-all accumulator inherently suboptimal.

To address this, we leverage the morsel-wise framework of AMS to introduce a hybrid accumulation mechanism (HYB) that dynamically selects the most suitable accumulator for each morsel. Specifically, we make this choice based on the aforementioned morsel-wise cf value, selectively choosing between two widely used accumulators: hash tables (HASH) [83, 84], which use linear probing and multiplicative hashing to reduce collisions, and expand-sort-compress (ESC) [29, 45], which relies on highly optimized radix sorting to alleviate common logarithmic sorting overheads.

The choice between these two accumulators is highly dependent on the morsel-wise cf. As ESC generates sorted output, it executes more instructions than HASH. However, this instruction overhead is not necessarily detrimental, particularly for low-cf morsels. For such morsels, the number of intermediate products is similar to the final number of nonzeros, leading to a moderate cost for merging and sorting. In contrast, HASH becomes more populated for low-cf morsels, and the overhead of hash computation and probing becomes more pronounced. For high-cf morsels, however, HASH is the better choice, as a massive number of intermediates are hashed into a few slots, greatly reducing potential cache misses. Conversely, ESC must pay a significant cache miss cost to merge a large volume of intermediate results into a small final output. Microbenchmark results in Figure 9 corroborate this finding: the ratio of ESC's LLC misses to HASH's is substantially higher on skewed G500 workloads with high cf than on non-skewed ER workloads.

Therefore, we empirically identify a cf threshold, using ESC for morsels with a cf below this value and opting for HASH for morsels above it [12]. This morsel-wise selection also offers an architectural benefit: selecting the accumulator on a per-morsel basis alleviates the instruction-level branch-misprediction costs that finer-grained (e.g., per-row) selection would incur.

It is worth noting that by including HASH, our HYB approach may produce a non-sorted result matrix, which is acceptable for applications like relational queries and graph analytics that do not require sorted output [58, 83]. However, if sorted output is necessary, HYB can be easily adapted: instead of choosing between HASH and ESC, it can be configured to choose the efficient strategy from a set of sorted accumulators, such as ESC, HEAP [84], SPA [42], or even a hash-then-sort approach [26], to ensure a fully sorted output. Consequently, this adaptive approach ensures each morsel is processed with the most suitable accumulator, leading to better overall SpGEMM performance regardless of the output ordering requirement.

3.5 Implications and Broader Applicability

Having presented AMS's components, we now step back to consider what they imply beyond the SpGEMM kernel itself. AMS highlights an important interplay between load balance and cache efficiency. Static partitioning preserves locality but leaves cores idle under skew, thereby underutilizing memory bandwidth; dynamic scheduling keeps cores busy but raises cache contention once the working sets of co-executing morsels exceed the shared cache. Effective scheduling must therefore maintain enough concurrency to saturate the memory system while limiting avoidable cache traffic, which is what AMS achieves by scheduling morsels from cheap metadata. This balance is particularly important for bandwidth-expanded memory [116], where utilizing the additional bandwidth requires enough cores to issue concurrent memory requests. By keeping cores busy while reducing cache interference, AMS sustains memory bandwidth beyond the local-DRAM peak (§ 4.4).

The same design principle can apply beyond SpGEMM when lightweight metadata is available to estimate each morsel's processing cost and demand on the shared cache. For SpGEMM, the symbolic phase provides this information directly. Other operators can obtain approximate metadata through a preliminary statistics pass [7, 25] or sampling [113, 115]; exact estimates are unnecessary because CALB uses only the relative ordering of morsels by cache demand and SMTact only tests whether a threshold is crossed. This creates opportunities to apply contention-aware morsel scheduling to other operators over relational and graph data with skewed, cache-resident state [3, 31, 43, 58, 103]. For example, a morsel-driven engine [33, 73] could use partition-size estimates to approximate cache demand when scheduling hash-aggregation or hash-join tasks over skewed keys.

A direct and notable extension of AMS, beyond SpGEMM and the relational query and graph operators it underpins, is to apply it atop other sparse matrix multiplication kernels, e.g., sparse matrix-dense

[12]We find that a cf of 1.5–1.7 provides a robust threshold for selecting the appropriate accumulator.

Table 1: Summary of 20 representative matrices evaluated.

Matrix	row(A)	nnz(A)	gini(A)†	flop‡	nnz(A^2)	cf
hugetric-00020	7.13M	21.37M	0.0003	64.08M	44.61M	1.4363
GL7d18	1.96M	35.60M	0.0395	890.29M	843.47M	1.0555
333SP	3.72M	22.22M	0.0559	134.79M	71.97M	1.8730
NLR	4.17M	24.98M	0.0736	152.77M	82.02M	1.8626
M6	3.51M	21.01M	0.0742	128.49M	68.99M	1.8625
AS365	3.80M	22.74M	0.0765	138.91M	74.62M	1.8616
auto	448.70K	6.63M	0.1016	101.25M	33.06M	3.0634
delaunay_n23	8.39M	50.34M	0.1220	316.97M	173.67M	1.8251
nv2	1.46M	37.48M	0.1401	1.28B	292.14M	4.3475
Freescale1	3.43M	17.06M	0.1858	90.33M	38.31M	2.3580
gsm_106857	589.45K	21.76M	0.2240	947.26M	132.40M	7.1546
amazon-2008	735.33K	10.32M	0.2856	218.17M	63.16M	3.4546
webbase-1M	1.01M	3.11M	0.3455	1.37B	1.31B	1.0475
vas_stokes_2M	2.15M	65.13M	0.3771	6.95B	1.17B	5.9452
rel9	9.89M	23.67M	0.4016	2.10B	2.04B	1.0300
vas_stokes_1M	1.10M	34.77M	0.4081	4.39B	721.63M	6.0776
patents	3.78M	14.98M	0.6585	178.30M	132.12M	1.3496
cit-Patents	3.78M	16.52M	0.6844	239.46M	162.35M	1.4749
NotreDame_www	325.73K	929.85K	0.7825	418.01M	288.98M	1.4465
web-NotreDame	325.73K	1.50M	0.8523	503.57M	293.20M	1.7175

† Gini coefficient [70] of the row-wise nonzeros in A.
‡ Only multiplications are counted, as additions are equal in number.

matrix multiplication (SpMM) and sparse matrix-vector multiplication (SpMV), which multiply a sparse matrix by a dense operand. Their output is dense and of fixed size, which renders HYB unnecessary and makes morsel formation by a target result size meaningless; we instead group consecutive rows by a target *nnz* of the left-hand operand. Although the skew thereby reduces to a single dimension (flop count), load imbalance and cache contention persist, as morsels containing many short rows and those containing a few long rows differ markedly in data reuse [39, 119]. Therefore, CALB and SMTact can address these challenges in SpMM and SpMV using the same principles as in SpGEMM [75]. Since most SpMM and SpMV solutions likewise follow a Gustavson-style row-wise formulation, AMS can be adapted to these kernels in a straightforward manner.

4 Experimental Evaluation

This section evaluates AMS in addressing multi-dimensional skew and on-chip cache contention. We begin with the experimental setup (§ 4.1), then benchmark AMS on matrix squaring across diverse workloads (§ 4.2) and dissect CALB and SMTact to validate their contention alleviation (§ 4.3). We further evaluate AMS on a bandwidth-expanded CXL system to show improved bandwidth utilization (§ 4.4). We then demonstrate its broad applicability on other sparse matrix multiplication kernels (§ 4.5), on relational join-aggregate and join-project queries (§ 4.6), and on graph-analytic workloads (§ 4.7), including PathSim meta-path construction [103], breadth-first search, and triangle counting. We close with a discussion (§ 4.8) that distills the key findings across these workloads and platforms, and relates them back to the design rationale of AMS.

4.1 Experimental Setup

We conduct experiments on a single-socket machine running Linux kernel 6.11.0. This system is equipped with an Intel Xeon® Gold 6430 CPU, which features 32 physical cores and supports 64 logical threads via simultaneous multithreading (SMT) [13] . Each core includes a 48 KB L1 data cache, a 32 KB L1 instruction cache, and a 2 MB L2 cache, while all cores share a 60 MB last-level cache (LLC). The system comprises 192 GB of DDR5 DRAM, providing 179.50 GB/s of memory bandwidth. We also conduct experiments on a CXL-augmented memory system to validate the generalizability of AMS, with the setup details deferred to § 4.4.

Following prior settings [5, 18, 20, 21, 83], we conduct our primary performance analysis using R-MAT matrix squaring workloads [22, 40, 65]. We employ three representative sets of R-MAT matrices to model different workload characteristics: ER ($a = b = c = d = 0.25$), SSCA ($a = 0.60, b = c = 0.13, d = 0.14$), and G500 ($a = 0.57, b = c = 0.19, d = 0.05$), which represent non-skewed, skewed, and highly skewed workloads, respectively. All R-MAT generators support control over size and density via two parameters: *scale* (determining the number of rows, i.e., 2^{scale}) and *edgefactor* (defining the average nonzeros per row). We sweep *scale* over 19–21 for ER, 18–20 for SSCA, and 17–19 for G500, with *edgefactor* over 2–32 in all three. For SSCA and G500 workloads, higher *edgefactors* amplify multi-dimensional skew and cache contention (§ 2.2). Additionally, we evaluate SpGEMM performance using real-world workloads from the SuiteSparse Matrix Benchmark [30] to validate the robustness of AMS. We perform matrix squaring on 119 SuiteSparse matrices, with row counts ranging from 100K to 10M and nonzero counts from 146K to 316M. We choose this range so that even the smallest matrix exceeds the 2 MB private L2 cache capacity [14]. This ensures that all workloads exercise the cache hierarchy beyond private L2, while keeping the largest matrices small enough to complete on our platform without exhausting memory. For a clear presentation, we present results for 20 representative matrices (Table 1), ordered by their Gini coefficient [15], as other matrices show similar trends. Beyond matrix squaring, we evaluate AMS on SpMM and SpMV (§ 4.5), relational join-aggregate and join-project queries (§ 4.6), and graph-analytic workloads, including PathSim meta-path construction, multi-source breadth-first search, and triangle counting (§ 4.7). The corresponding workloads and experimental setups are detailed in these subsections.

We integrate our AMS framework into two prevalent Gustavson-based SpGEMM algorithms: the non-sorted HASH algorithm [83, 84] and the sorted ESC algorithm [12, 29]. These are referred to as AMS-HASH and AMS-ESC, respectively [16] . Moreover, we implement our hybrid accumulation mechanism (HYB) as AMS-HYB, which adaptively selects between HASH and ESC accumulators. We empirically tune the parameters for AMS-based approaches, configuring morsel size at half the L2 cache size, and distributing threads using a 60%-25%-15% ratio across groups #0, #1, and #2, respectively, for optimal CALB performance.

[13] We disable Intel Speed Select Technology - Base Frequency to ensure equal performance for all physical cores [109].
[14] Storing a matrix of n rows and nnz nonzeros in CSR with 8-byte indices and double-precision values takes $8(n+1) + 16\,nnz$ bytes, corresponding to approximately 3 MB at the lower end of this range and 5.1 GB at the upper end.
[15] The Gini coefficient measures inequality in the row-wise nonzero distribution; a value of 0 indicates uniform distribution, while values approaching 1 indicate extreme skew. Coefficients above 0.30 generally indicate skewed workloads [30, 70].
[16] Sorted algorithms generally incur more overhead to produce row-wise sorted output.

Table 2: Summary of all evaluated SpGEMM methods

Method	Category	Sorted	Description
Algorithmic baselines (built upon by our AMS methods)			
`HASH [84]`	Baseline	✗	Hash table accumulator with flop-count-based scheduler.
`ESC [29]`	Baseline	✓	Radix-sort (expand-sort-compress) accumulator with flop-count-based scheduler.
External / industry competitors			
`HEAP [84]`	Competitor	✓	Heap accumulator with flop-count-based scheduler.
`PB [45]`	Competitor	✓	Outer-product SpGEMM flow with flop-count-based scheduler.
`MKL [60]`	Competitor	✗	Intel oneMKL SpGEMM — non-sorted variant.
`MKLS [60]`	Competitor	✓	Intel oneMKL SpGEMM — sorted variant.
Proposed AMS-based methods			
`AMS-HASH`	Proposed	✗	AMS approach using a HASH accumulator.
`AMS-ESC`	Proposed	✓	AMS approach using an ESC accumulator.
`AMS-HYB`	Proposed	✗	AMS with per-morsel adaptive HASH/ESC selection.

We compare our methods against several state-of-the-art baselines (see Table 2 for an overview). These include: (1) the same HASH and ESC algorithms driven by a state-of-the-art flop-count-based scheduler; (2) the sorted HEAP algorithm [83, 84], which exploits the implicit column ordering of input matrices to reduce sorting overhead at the cost of increased memory usage; (3) the outer-product-based PB method [45]; and (4) the de facto industry solution, Intel MKL [60], including both its sorted (MKLS) and non-sorted (MKL) variants. While other SpGEMM algorithms, such as SPA [42] and SaSpGEMM [50], were also evaluated, we omit their results for conciseness as they consistently underperformed.

All implementations are compiled using `GCC-11.5.0` with the `-O3` optimization flag and parallelized with OpenMP [28]. Baselines are manually tuned for optimal performance, which includes enabling SMT for non-skewed workloads and disabling it for skewed ones. Our AMS-based methods are tuned with equal care, sweeping the thread group sizes, the low/high-cf boundary, and the SMTact threshold. As in prior works, throughput, measured in GFLOPS (giga floating-point operations per second), serves as the primary evaluation metric. Unless a measurement is explicitly labeled as numeric-phase profiling, all reported runtimes and throughputs cover the complete SpGEMM pipeline, i.e., the symbolic phase, prefix-sum computation and output allocation, AMS preparation, the numeric phase, and output finalization. All reported results are averaged over 10 runs [17].

4.2 Matrix Squaring Performance Evaluation

Figure 10 presents the matrix squaring performance on R-MAT and SuiteSparse workloads, and shows that our AMS-based algorithms consistently achieve high performance across this broad spectrum. Specifically, the proposed AMS-HYB is generally the superior solution: it maintains performance comparable to AMS-HASH in skewed workloads (SSCA and G500) while remaining as competitive as AMS-ESC in non-skewed, high-*edgefactor* workloads (ER). This outcome validates both the effectiveness of our HYB design and the soundness of our cf-based criterion for adaptively selecting the best morsel-wise accumulator for efficient multiplication.

Beyond the superior performance of AMS-HYB, the AMS framework itself significantly boosts both its non-sorted (AMS-HASH) and sorted (AMS-ESC) variants. These methods perform comparably to their respective baselines on ER matrices but deliver a substantial performance advantage on the skewed SSCA and G500 matrices. For the non-sorted variant, the gains are larger on the more skewed G500 workloads than on SSCA at matched *scale* and *edgefactor*, where the two generators differ only in their R-MAT probabilities. For example, at *scale* 19 and *edgefactor* 2, AMS-HASH achieves speedups of 1.44× on G500 and 1.23× on SSCA over the HASH baseline. The sorted AMS-ESC variant also benefits on skewed workloads, although its gains are smaller than those of AMS-HASH, as discussed below. Together, these results demonstrate that AMS's dynamic scheduling, when paired with appropriately sized morsels, effectively addresses multi-dimensional skew without sacrificing performance on non-skewed workloads. The strong results on ER matrices (Figure 10(a)–(c)) also validate the effectiveness of SMTact, which selectively enables SMT in low-contention scenarios to enhance computational throughput, a critical factor in non-skewed workloads.

Comparing the impact of AMS across different algorithms, we observe that the performance improvement is more pronounced in HASH than in ESC. This is because ESC relies on radix sort, which has a complexity of $O(\omega \cdot n)$ and correlates directly with the flop count. Although HASH-based methods also exhibit $O(n)$ complexity in hash table lookups, their efficiency is more susceptible to "clustering" effects, making flop-count-based models less effective in accurately estimating cost and balancing workloads. In contrast, our AMS approach is agnostic to the underlying SpGEMM algorithm. For both non-sorted and sorted algorithms, AMS achieves performance gains of up to 81.01% and 36.03%, respectively, demonstrating its general applicability across algorithmic variants.

To further illustrate the source of these gains, Figure 11 drills down into the per-thread elapsed time, comparing AMS-based methods against their respective baselines on a representative skewed workload. The baseline HASH and ESC exhibit significant thread-level runtime variance, confirming that flop-count-based scheduling fails to balance the actual workload under skew. In contrast, AMS-based variants achieve substantially more uniform per-thread runtimes, demonstrating that our scheduling framework effectively addresses the load imbalance problem inherent in skewed workloads.

Despite these advantages, Figure 10 shows that AMS-based algorithms may occasionally underperform their flop-count-based counterparts in highly sparse cases (e.g., G500 with *scale* 17 and *edgefactor* 2), primarily due to overhead from cf-based morsel sorting. Nevertheless, this penalty diminishes as matrix density increases, underscoring the lightweight nature of AMS in addressing load imbalance.

Turning to the competing methods, a consistent performance ranking emerges across the tested configurations. The baseline HASH and ESC algorithms trail their AMS-enhanced counterparts, and the de facto industry solution, Intel MKL, falls behind those baselines in turn: AMS-HYB surpasses the non-sorted MKL by up to 3.83× and the sorted MKLS by up to 8.45×, while AMS-ESC delivers

[17] The source code, data, and/or other artifacts have been made available at https://github.com/fukien/contention-aware-spgemm.

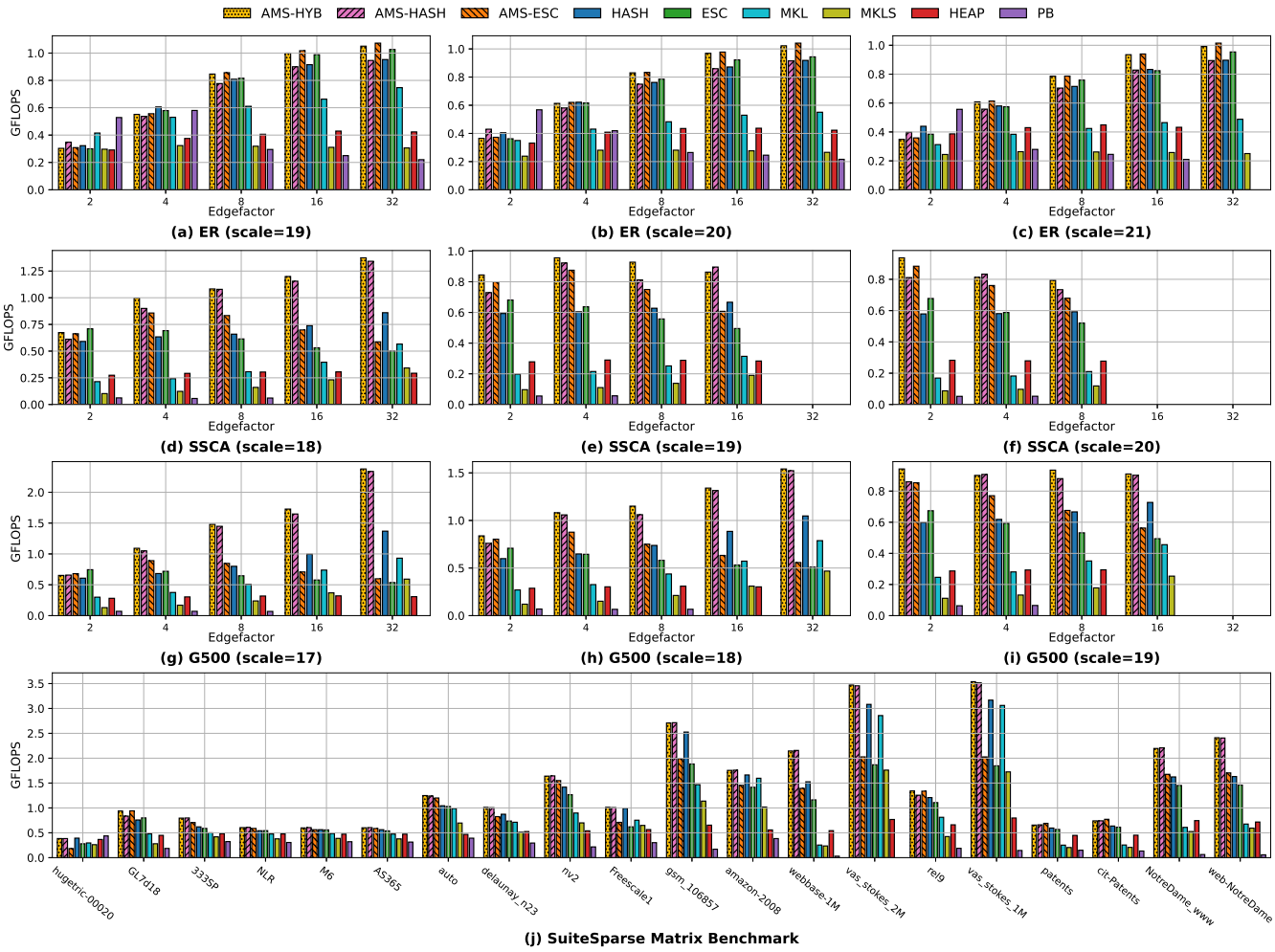


Figure 10: SpGEMM performance evaluation. Subfigures (a)–(i) present results for R-MAT matrices (ER, SSCA, G500) across varying scales and *edgefactor* values, representing changes in matrix dimension and density, respectively. Subfigure (j) displays results for 20 representative matrices from the SuiteSparse collection, ordered on the x-axis by their Gini coefficient (ascending), which correlates with increased workload skew. Empty bars signify out-of-memory (OOM) failures.

up to 7.66× over MKLS. HEAP and PB rank last, with more unfinished runs owing to their notably larger memory demands. The one exception is PB on highly sparse non-skewed workloads (ER matrices, *edgefactor* 2), where its outer-product-based formulation maximizes data reuse [45], an advantage that quickly diminishes at higher *edgefactor* values and thus implies limited applicability across diverse workloads. These observations underscore the need for adaptive SpGEMM processing, as no single formulation dominates across the density spectrum.

The SuiteSparse results (Figure 10(j)) show the same performance ranking on real-world matrices as on the synthetic R-MAT matrices. AMS-HYB outperforms every competitor on all 20 matrices, except for one matrix against HASH and one against PB. Consistent with the R-MAT results, AMS-HASH and AMS-ESC outperform their flop-count-based counterparts on 19 of the 20 matrices, by up to 47.46% and 25.92%, respectively. Compared with the other methods, AMS-HYB achieves geometric-mean speedups of 1.16× over HASH, 1.37× over ESC, 1.81× over MKL, 2.24× over HEAP, and 2.41× over MKLS. Its largest advantage is over PB, with a geometric-mean speedup of 6.08×; PB is also the only method that fails to finish on one of the 20 matrices. These results on real-world matrices show that AMS's benefits extend beyond the synthetic R-MAT workloads.

4.3 Dissecting AMS: CALB and SMTact

4.3.1 Effectiveness of CALB. We now examine the performance benefits of our CALB approach. Designed to alleviate cache contention while maintaining load balance, CALB is evaluated against two alternative scheduling techniques also applicable to our cf-sorted morsels: (1) a bidirectional greedy scheduling scheme (BGS), where threads split into two equal-sized groups processing morsels

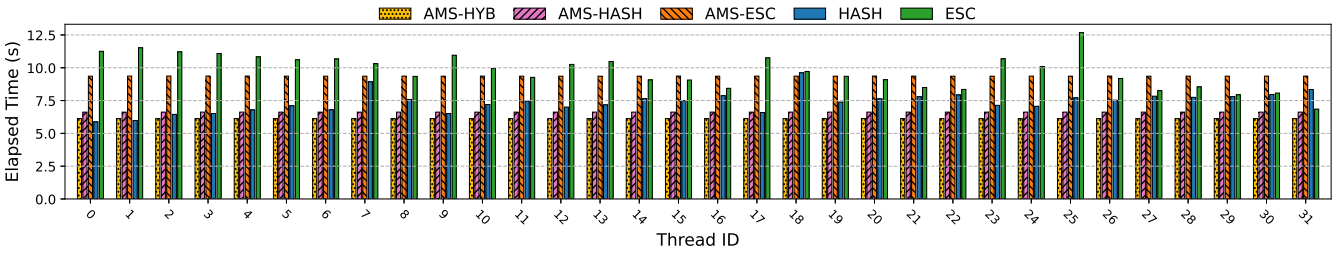


Figure 11: Per-thread elapsed time of the numeric phase comparison of AMS-based variants (AMS-HYB, AMS-HASH, AMS-ESC) and their baseline counterparts (HASH, ESC) on a G500 skewed matrix at *scale* 19 and *edgefactor* 8.

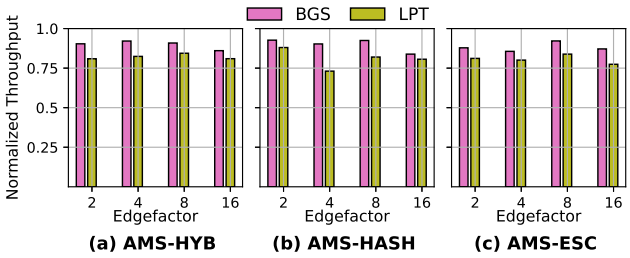


Figure 12: Throughput comparison of BGS and LPT scheduling strategies against CALB. The y-axis shows throughput on G500 (scale=19) workloads, normalized to CALB.

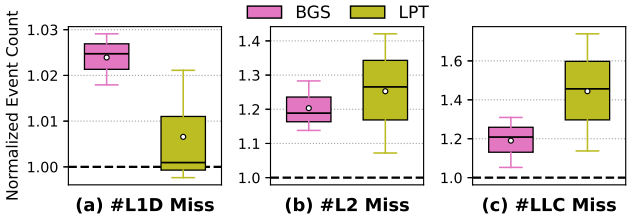


Figure 13: Distribution of the L1D, L2, and LLC misses incurred by the BGS and LPT scheduling strategies on G500 workloads (scale=19), each normalized to the corresponding CALB-based execution (dashed line at 1.0). Every box aggregates the samples of all three AMS variants, i.e., AMS-HYB, AMS-HASH, and AMS-ESC. Within each box, the solid line marks the median, the white circle the mean, and the whiskers extend to the most extreme samples within 1.5× the interquartile range. (a) L1D misses. (b) L2 misses. (c) LLC misses.

from opposite ends toward the center, and (2) the classical longest-processing-time-first strategy (LPT) [44], in which all threads start from the high-cf end and progress toward the low-cf end.

Figure 12 reports the throughput of BGS and LPT, normalized to our CALB method, for three AMS-based algorithms on G500 workloads at *scale* 19. The results show that CALB consistently outperforms both alternatives, achieving 7.90%–19.26% and 8.53%–36.80% higher throughput than BGS and LPT, respectively. While BGS also aims to mitigate contention by avoiding excessive thread concentration on high-cf regions, our three-group division strategy in CALB proves more effective at reducing cache pressure, especially at the high-cf end. In contrast, LPT performs poorly precisely because it schedules all threads to handle the most demanding, high-contention tasks simultaneously, thereby exacerbating on-chip cache contention.

This explanation rests on cache contention, so we verify it by direct measurement rather than by inference from throughput. Figure 13 profiles the L1D, L2, and LLC misses incurred by BGS and LPT with hardware performance counters, again normalized to CALB on the same G500 workloads, with every box aggregating the samples of all three AMS variants. The counters are collected separately from the timing measurements and averaged over 10 runs. Since all three policies perform the same logical SpGEMM work using the same accumulator and input, their event counts are directly comparable. Figure 13(a) shows that the three policies incur nearly identical L1D miss counts, with all measurements within 3% of CALB. This is expected because the private L1D cache is too small to hold a morsel's working set, leaving limited scope for dispatch order to improve locality at this level. The differences emerge in the deeper cache levels, where concurrently executing morsels compete for capacity. BGS and LPT incur 1.14–1.28× and 1.07–1.42× as many L2 misses as CALB, respectively (Figure 13(b)). LPT also incurs the most LLC misses and exhibits greater variation than BGS (Figure 13(c)). These cache-miss trends are consistent with the throughput results: LPT incurs the most misses and delivers the lowest throughput, supporting cache contention as a key factor beyond load balance alone.

This validates the rationale behind CALB: by dispersing threads across the cf spectrum instead of concentrating them at the high-cf end, CALB reduces the aggregate cache pressure of concurrently processed morsels, confirming that effective load balancing must account for cache contention. Notably, LPT has been a seminal scheduling strategy for over 50 years [44], and our findings highlight the need to revisit and refine classical techniques with the idea of cache pressure alleviation.

We further compare AMS against two widely adopted OpenMP load-balancing strategies: dynamic and guided scheduling [16, 89]. These policies schedule a unit of work different from our morsel, so they replace AMS's scheduling as a whole, and both Figure 14 and Figure 15 are accordingly normalized to AMS. For a fair evaluation,

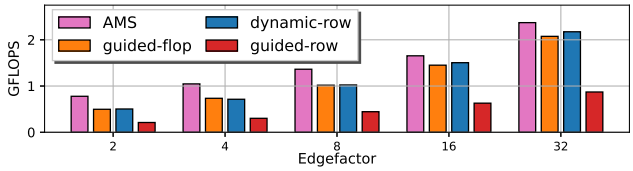


Figure 14: Hashing-based SpGEMM performance on G500 matrices (scale=17) compared to OpenMP dynamic (-row) and guided (-row and -flop) scheduling policies.

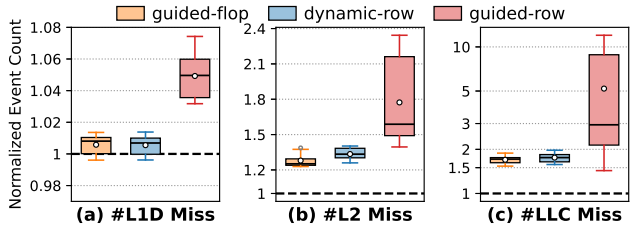


Figure 15: Distribution of the L1D, L2, and LLC misses incurred by the OpenMP dynamic-row, guided-flop, and guided-row scheduling policies of Figure 14, each normalized to the corresponding AMS-based execution (dashed line at 1.0). Within each box, the solid line marks the median, the white circle the mean, and the whiskers extend to the most extreme samples within 1.5× the interquartile range. Note the logarithmic y-axis in (b) and (c). (a) L1D misses. (b) L2 misses. (c) LLC misses.

we implemented dynamic scheduling using an empirically determined optimal morsel size of 128 consecutive rows (dynamic-row). Guided scheduling is tested with a minimum morsel set by either 128 rows (guided-row) or a 128-floating-point operation (FLOP) count (guided-flop); both configurations were chosen for the purpose of preserving spatial locality.

As Figure 14 illustrates, AMS achieves higher throughput across all comparisons: 11.87%–48.01% over dynamic-row, 12.89%–47.13% over guided-flop, and a substantial 2.79×–3.59× over guided-row. The two margins have different origins, and Figure 15, which profiles the same three policies with hardware performance counters, distinguishes them.

Guided-row is the outlier on both figures. OpenMP guided scheduling progressively reduces its chunk size, which, similar to LPT [44], degrades spatial locality during later processing stages, and the profile shows exactly that: its median rises from about 1.05× at L1D to about 1.6× at L2 and 2.9× at the LLC, with individual measurements exceeding 10× AMS and by far the widest spread of the three policies. This margin is attributable to morsel-wise dynamic scheduling itself (§ 3.1), which keeps consecutive rows together and thereby preserves the locality that guided-row progressively gives up; that alone is enough to outperform guided-row comfortably. This result underscores that spatial locality is a key performance factor, and that it must be preserved together with load balancing.

Dynamic-row and guided-flop avoid this pitfall, as their work units keep consecutive rows together and they distribute load as effectively as AMS does. Yet both still trail AMS, and the cache

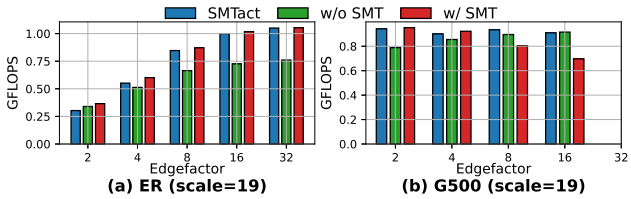


Figure 16: Effectiveness of SMTact in managing SMT to improve AMS-HYB performance on non-skewed (ER) and skewed (G500) workloads.

profiles show where the difference arises. At the private L1D, where a difference in spatial locality would first become visible, both remain within roughly 1% of AMS. The policies diverge only at the levels that concurrently executing morsels share, with median counts rising to 1.27–1.34× at L2 and to 1.7–1.8× at the LLC, a ranking consistent with their relative throughput in Figure 14. Since locality is comparable across the three, the remaining source of this gap is inter-thread cache contention, which CALB alleviates by co-scheduling high-cf and low-cf morsels. We therefore attribute the advantage of AMS over these two policies to CALB, and conclude that contention-aware scheduling is essential for effective parallel load balancing.

4.3.2 Effectiveness of SMTact. Our SMTact is designed as an adaptive SMT-enablement strategy by balancing the trade-off between increased computational capability and elevated cache pressure. To evaluate its effectiveness, we compare SMTact against two fixed configurations: SMT always enabled (w/ SMT) and SMT always disabled (w/o SMT). The results, presented in Figure 16, include both non-skewed and skewed workloads. As illustrated, neither fixed configuration is uniformly viable: w/ SMT falls up to 23.9% short of the best configuration on the skewed G500 workloads, where the extra logical threads aggravate cache contention, whereas w/o SMT forfeits up to 28.6% on the non-skewed ER workloads that profit from the additional thread-level parallelism.

Since SMTact ultimately executes as one of these two configurations, its goal is not to surpass both in every case, but to automatically select a configuration at or close to the best. In our experiments, SMTact stays within 8.2% of the best fixed configuration in eight of the nine cases. The exception is the extremely sparse ER matrix at *edgefactor* 2, where SMTact falls 17.1% short of the best fixed configuration because the sorting overhead introduced by CALB is not yet amortized; this penalty is quickly offset as *edgefactor* or matrix scale increases. Given the wide variability in sparsity patterns in real-world SpGEMM workloads, SMTact thus offers a robust default that avoids the substantial degradation that either fixed policy incurs outside its sweet spot.

4.4 Evaluation on Bandwidth-Expanded Systems

We now evaluate AMS on a bandwidth-expanded memory system, where CXL boosts aggregate bandwidth but introduces higher access latency. As discussed in § 1, load imbalance can inadvertently amplify CXL's latency penalty on straggler threads, negating the bandwidth gain and making effective load balancing particularly

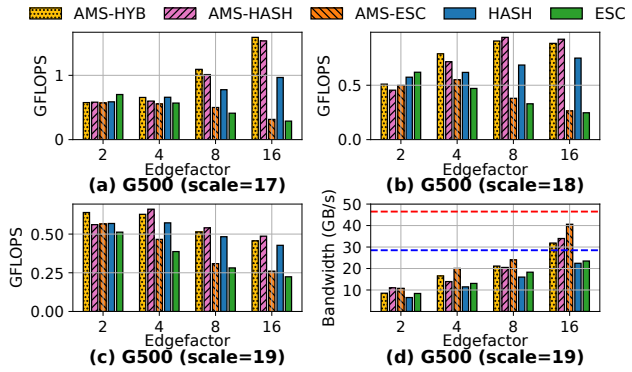


Figure 17: Performance of SpGEMM on G500 workloads. (a)–(c): Measured throughput at G500 scales 17, 18, and 19. (d): Sustained bandwidth measured for different algorithms for the scale-19 workload (the red and blue dashed lines indicate the peak bandwidth of the CXL-augmented memory system (46.50 GB/s) and host DRAM (28.50 GB/s), respectively).

critical. For this evaluation, the machine has the same CPU and OS configuration as our previous platform but features host DRAM of 28.50 GB/s bandwidth. Additionally, a CXL expansion memory of 20.95 GB/s bandwidth is installed, yielding a combined system bandwidth of 46.50 GB/s [57, 71, 117].

We evaluate AMS approaches against their vanilla counterparts using skewed (G500) workloads and report the results in Figure 17. As shown, AMS-based methods generally outperform the other methods (Figure 17(a)–(c)). Furthermore, the profiling results confirm that AMS effectively utilizes the additional CXL-augmented bandwidth: AMS-ESC sustains a bandwidth of up to 40.60 GB/s, far exceeding the DRAM-only peak (28.50 GB/s) (Figure 17(d)). In contrast, the bandwidth consumption of the vanilla counterparts does not exceed that of the host DRAM, indicating that AMS indeed improves bandwidth utilization by addressing load imbalance.

We also observe an interesting trend in Figure 17(a)–(c): while AMS-based methods remain competitive on skewed workloads, the performance gap between AMS and the vanilla baselines differs for G500 matrices at *scale* 17 versus those at *scale*s 18 and 19. Taking the HASH-based variants as an example, AMS-HASH outperforms HASH by 58.77% on the G500 matrix at *scale* 17, delivering 1.53 GFLOPS versus 0.97 GFLOPS (*edgefactor* 16). However, this advantage diminishes to 23.16% at *scale* 18, with a similar trend observed at *scale* 19.

This reduction in the performance gap is mainly attributed to the coincidental load balance occasionally achieved by the static flop-count-based scheme. It is the extent of this imbalance that largely determines the performance degradation of baselines relative to their AMS counterparts. As Figure 4 illustrates, computation and materialization times can exhibit opposing trends across threads. This is precisely the multi-dimensional skew identified in § 2.2, namely that computation and materialization costs trend in opposite directions, and that any single-metric scheme can only coincidentally balance them. In some cases, these opposing patterns may randomly offset each other, resulting in a seemingly balanced overall execution time despite the underlying skew. This balance, however, is merely incidental and heavily dependent on the platform's relative sensitivity to computation versus memory access. Consequently, it fails to provide consistent or portable load balance across different architectures.

Previous results on a 179.50 GB/s bandwidth memory corroborate this explanation (see Figure 10(g)–(i)). With this higher bandwidth, AMS-HASH achieves a 45.44% performance increase over HASH (1.52 GFLOPS vs. 1.05 GFLOPS) on the G500 matrix at *scale* 18, because the increased bandwidth significantly reduces the platform's vulnerability to materialization cost. Collectively, these results validate our claim that AMS provides a robust and portable solution to load balancing across diverse systems.

4.5 Evaluation on Other Sparse Matrix Kernels

Having established AMS's portability across memory systems, we now turn to its generalizability across kernels. Beyond SpGEMM, we evaluate AMS on other sparse matrix multiplication kernels, specifically, sparse matrix-dense matrix multiplication (SpMM) and sparse matrix-vector multiplication (SpMV). Since these kernels produce fixed-size dense output, we form morsels by grouping consecutive rows based on a targeted *nnz* count in the left-hand side matrix sub-block (see § 3.5), with each morsel sized at half the L2 cache capacity. The resulting morsels have disparate densities, enabling CALB and SMTact to alleviate cache contention as in SpGEMM.

We compare this AMS-based SpMM and SpMV implementation against the state-of-the-art Intel MKL library on both non-skewed ER and skewed G500 workloads, running on the same main-memory system as our default platform (§ 4.1), taking ER at *scale* 21 and G500 at *scale* 20 and sweeping the *edgefactor* from 2 to 64, so that the behavior at higher densities remains visible.

As shown in Figure 18(a)–(d), our method achieves on-par performance with MKL on non-skewed ER workloads while significantly outperforming it on skewed G500 workloads, with throughput gains of up to 57.25% and 3.51× for SpMM and SpMV, respectively. Figure 18(e)–(h) reports the sustained memory bandwidth of the same runs and shows where that gain originates: on the non-skewed ER workloads the two methods sustain comparable bandwidth, with SpMM approaching the 179.50 GB/s physical limit of local DRAM, whereas on the skewed G500 workloads AMS sustains substantially higher bandwidth than MKL at every *edgefactor*. This confirms that the advantage comes from the load balancing of AMS together with the on-chip cache contention it alleviates, exactly as in SpGEMM, where stragglers induced by skew leave bandwidth idle and concurrently executing dense morsels evict each other's working sets. By removing both, AMS converts bandwidth that the baseline wastes into useful work, which is what drives the throughput gain on these memory-bound kernels.

The bandwidth results also reveal a notable difference between SpMM and SpMV. SpMM sustains markedly higher bandwidth than SpMV throughout, as its dense right-hand operand offers a reuse that a single vector cannot, so SpMV wastes a far larger share of the available bandwidth, and the waste is worst under skew, where MKL sustains only a small fraction of the DRAM peak (Figure 18(h)). AMS substantially improves bandwidth utilization on skewed SpMV workloads, sustaining roughly three times the bandwidth of MKL.

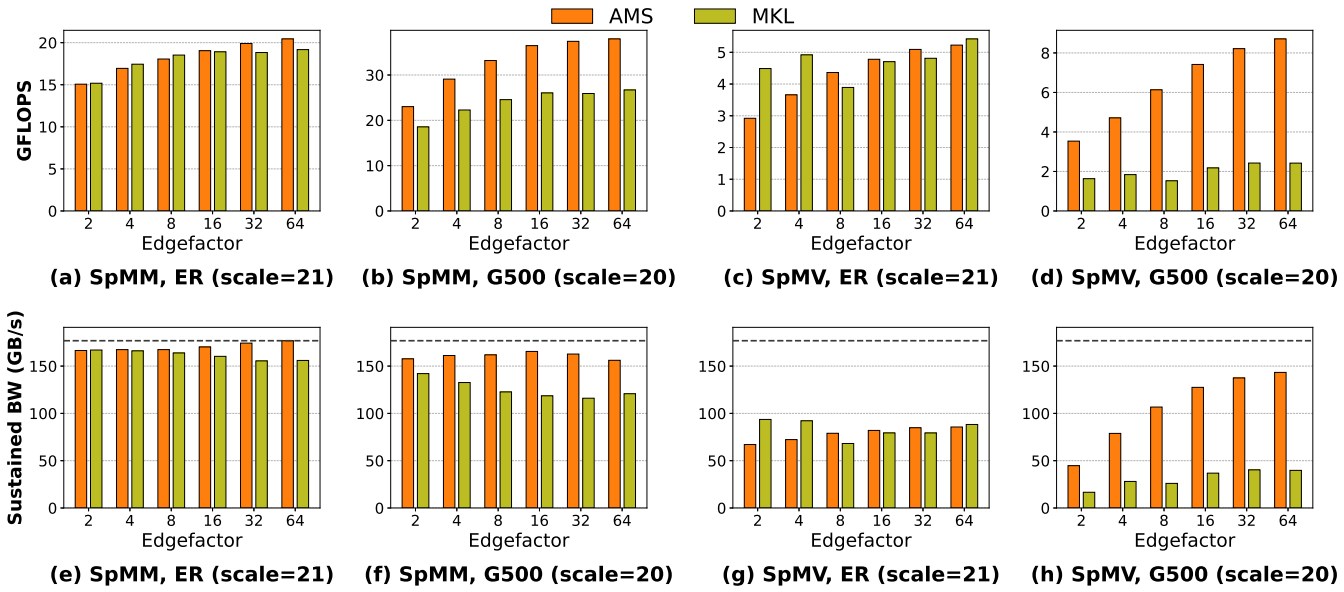


Figure 18: Performance of AMS versus MKL for SpMM and SpMV operations on ER and G500 workloads, reported as throughput in (a)–(d) and as sustained memory bandwidth in (e)–(h). The dashed line marks the 179.50 GB/s physical bandwidth of our platform's local DRAM.

At higher *edgefactors*, its sustained bandwidth approaches that of SpMM. These results show that skew-aware scheduling improves the performance of memory-bound kernels by enabling more effective use of the available memory bandwidth. Together, these results demonstrate the broad applicability and robust advantages of AMS's core scheduling principles across various sparse matrix multiplication kernels.

4.6 Join-Aggregate and Join-Project Query Evaluation

We now validate AMS on two representative relational query classes: join-aggregate and join-project, using the same datasets for both. These queries apply aggregation or projection after a join and are widely used in prior studies of matrix multiplication for relational query processing [31, 58, 101]. By modeling tables as sparse matrices, SpGEMM fuses the join with the subsequent aggregation or projection, avoiding materialization of the intermediate join result [1, 3, 64]. This is particularly beneficial for many-to-many joins and self-joins that produce large intermediate results.

We employ widely used datasets from prior works [31, 55, 58, 101], and also evaluate on an industrial OLAP ClickHouse benchmark [96] with two sampled workloads: CLH0, a uniform 5% sample of users, and CLH1, the top 5% of users by pageviews (see Table 3 for key statistics). These datasets span a wide range of skewness, from mildly to highly skewed, as quantified by the Gini coefficient in Table 3. Notably, the uniform sample exhibits a higher Gini coefficient than the top-user sample (0.59 versus 0.39), because selecting only the heaviest users excludes the low-activity tail, whereas uniform sampling largely preserves the original population's skew. As for the query form, we perform self-join (i.e., $R = S$) with aggregation in the form:

```
SELECT R.user, S.user, SUM(R.val * S.val)
FROM R, S
WHERE R.item = S.item AND R.user <> S.user
GROUP BY R.user, S.user
```

The corresponding join-project query replaces the aggregation with a projection over the same self-join:

```
SELECT DISTINCT R.user, S.user
FROM R, S
WHERE R.item = S.item AND R.user <> S.user
```

Both queries measure user-wise similarity, where "item" and "val" represent different attributes and interaction values depending on the scenario, e.g., URL/click-count in ClickHouse (CLH0 and CLH1) and movie/rating in MovieLens (MVLN). The aggregate query form is particularly challenging, as the aggregated values increase the demand for memory, and it is not amenable to existing aggregate query rewriting techniques [23, 34–36, 81, 114, 115, 120, 121].

Relational join queries do not require ordered output by default, and sorting the output within each row adds substantial overhead, as shown in § 4.2 and reported in prior SpGEMM studies [83, 84]. We therefore conduct this evaluation in the non-sorted regime, comparing AMS-HYB with: (1) DIM3 [58], the state-of-the-art join-aggregate method based on SpGEMM; (2) NPHJ-AGG, where a fast hash table [97] performs aggregation on-the-fly alongside the probing phase of non-partitioned hash join (NPHJ) [10, 54, 95], also bypassing intermediate materialization; (3) PHJ-AGG, where we use the cache-efficient partitioned hash join [6–8, 17] to perform the join and materialize the intermediate result, and after that we use the same efficient hash table [97] as NPHJ-AGG to perform the

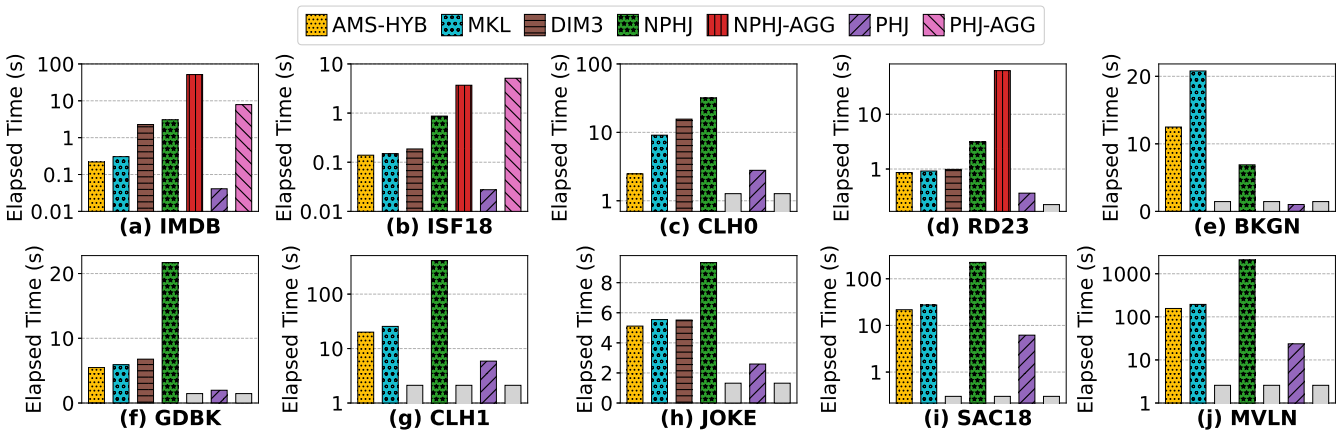


Figure 19: Execution time comparison across diverse join-aggregate workloads; gray bars with "N/A" denote algorithms that run out of memory on those specific workloads; note that vertical axes use a logarithmic scale to capture the wide range of execution times, except for subfigures (e), (f), and (h).

Table 3: Summary of the 10 relational workloads used for both join-aggregate and join-project evaluation.

Dataset	$\|user\|$	$\|tuple\|$	Gini	$\|OUT_J\|$	$\|OUT_A\|$	$\|OUT_J\|/\|OUT_A\|$
IMDB [59]	733.47K	7.52M	0.3453	214.77M	176.91M	1.2140
ISF18 [85]	1.82K	1.03M	0.5586	327.58M	3.29M	99.6586
CLH0 [96]	850.00K	2.90M	0.5858	2.43B	2.34B	1.0415
RD23 [99]	6.72K	3.91M	0.5276	4.61B	45.02M	102.3613
BKGN [69]	350.33K	5.15M	0.7242	11.43B	6.77B	1.6892
GDBK [124]	53.42K	5.98M	0.1293	19.63B	2.51B	7.8261
CLH1 [96]	850.00K	24.05M	0.3862	20.34B	11.41B	1.7823
JOKE [58]	23.50K	1.71M	0.1709	32.65B	552.25M	59.1219
SAC18 [68]	104.66K	10.00M	0.6988	68.52B	5.94B	11.5325
MVLN [58]	138.49K	20.00M	0.5807	269.61B	16.80B	16.0477

1. $|user|$ and $|tuple|$ are equivalent to the matrix row count and total nnz count, respectively, reflecting the relational-to-sparse-matrix duality; Gini captures the user-wise nnz distribution [70].
2. $|OUT_J|$ and $|OUT_A|$ represent the cardinalities of the join result and the final aggregated result, respectively; the projected result has the same cardinality.

Table 4: Summary of all evaluated join-aggregate query processing approaches

Method	In-place Agg.*	Description
Traditional join followed by aggregation		
`PHJ-AGG [95, 97]`	✗	Radix partitioned hash join with aggregation; deduplication via fast hash table [58, 97].
`NPHJ-AGG [13, 97]`	✓	Non-partitioned hash join; aggregates computed during probing to skip the final deduplication pass [58, 97].
Join-alone algorithms (no aggregation; for comparison only)		
`PHJ [95]`	—	Radix partitioned hash join; serves as the join backend of PHJ-AGG. Does not produce aggregation results.
`NPHJ [13]`	—	Non-partitioned hash join; serves as the join backend of NPHJ-AGG. Does not produce aggregation results.
State-of-the-art join-aggregate techniques		
`DIM3 [58]`	✓	Hybrid sparse/dense join-aggregate query algorithm; intersection-free partitioning eliminates deduplication.
Industry SpGEMM competitors		
`MKL [60]`	✓	Intel oneMKL SpGEMM (non-sorted variant); aggregation replaces deduplication inside the accumulator.
Proposed AMS-based approaches		
`AMS-HYB`	✓	AMS scheduling with hybrid HASH/ESC selection; eliminates the separate deduplication pass.

* *In-place Agg.* denotes whether aggregation is performed on-the-fly alongside the join, or both operations are directly merged into one.

aggregation [18] ; (4) the industrial SpGEMM method: MKL with non-sorted output. We refer to NPHJ-AGG and PHJ-AGG collectively as the "join-then-aggregate" methods, as both evaluate the query in a traditional way, performing the aggregation on top of an explicit join rather than fusing the two as the matrix-based methods do. For reference, we also include NPHJ and PHJ as join-only baselines (which do not materialize the intermediate results). All evaluated methods are summarized in Table 4 for a clear reference.

As for the join-project setting, we use the same set of methods but replace the aggregation with a projection. A projection no longer references the value domain, so both the matrix-based and the traditional "join-then-project" methods can omit the value computation altogether, avoiding the memory traffic and arithmetic cost associated with processing these values, and every method that produces the query output is correspondingly faster than in the join-aggregate setting above. For the matrix-based methods, i.e., AMS-HYB, MKL, and DIM3, the value array of the output CSR matrix is thus neither allocated nor populated, and the projection is read directly from the sparsity structure of the matrix product. Moreover, join-project corresponds to a Boolean matrix product, so only the existence of each output nonzero matters. Following prior work [3, 58], we therefore enable early termination in AMS-HYB and DIM3, stopping the accumulation for each output cell as soon as it is known to be nonzero. In contrast, join-aggregate must sum all contributions to each output cell. It should be noted that we are not able to apply this mechanism to MKL, as the proprietary

[18] We do not have PHJ-AGG perform the on-the-fly aggregation as NPHJ-AGG, because on-the-fly aggregation would incur substantial memory traffic to the aggregation hash table, breaking the cache efficiency that partitioning is designed to deliver [7, 8, 80].

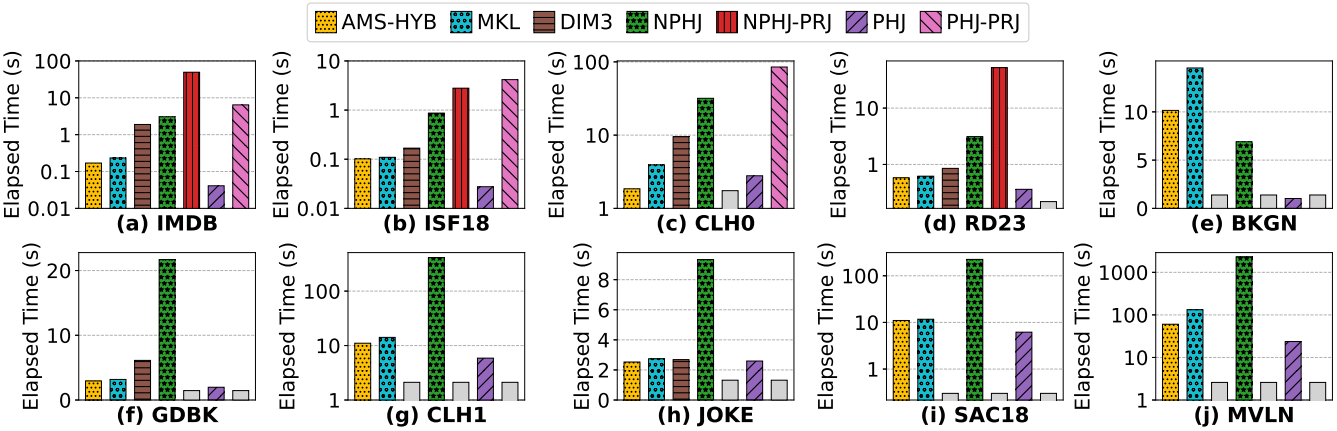


Figure 20: Execution time comparison across diverse join-project workloads; gray bars with "N/A" denote algorithms that run out of memory on those specific workloads; note that vertical axes use a logarithmic scale to capture the wide range of execution times, except for subfigures (e), (f), and (h).

Table 5: Tuple-to-CSR conversion cost, covering both operands of each query, and its share of the total AMS-HYB runtime under the two query classes.

Dataset	Conversion (s)	Share (%)	
		Join-Aggregate	Join-Project
IMDB	0.0126	5.7536	7.3995
ISF18	0.0043	3.0764	4.1958
CLH0	0.0087	0.3534	0.4729
RD23	0.0064	0.7476	1.1090
BKGN	0.0128	0.1025	0.1260
GDBK	0.0097	0.1764	0.3282
CLH1	0.0679	0.3391	0.6139
JOKE	0.0049	0.0962	0.1948
SAC18	0.0156	0.0720	0.1422
MVLN	0.0488	0.0313	0.0807

library exposes only a numerical product. For the traditional "join-then-project" methods, denoted NPHJ-PRJ and PHJ-PRJ, no value payload is carried or combined, and the two mirror their aggregate counterparts: NPHJ-PRJ projects on-the-fly alongside probing as NPHJ-AGG does, whereas PHJ-PRJ projects only after the intermediate result is materialized, for the same reason as PHJ-AGG.

It is noteworthy to mention that in this evaluation, the table inputs are in the original row-wise tuple format ⟨row-index, column-index, value⟩, e.g., ⟨user, URL, click count⟩ for the ClickHouse workloads, which is equivalent to the coordinate format (COO) in sparse matrix storage [37, 67, 93] [19]. The hash-join methods operate on these tuples directly, whereas the matrix-based methods require CSR and therefore incur a preliminary pass of data format conversion. We perform this tuple-to-CSR conversion with a parallel pass of histogram, prefix sum, and scatter, following the common practice of mainstream sparse-matrix libraries [46, 111], and charge its time to all matrix-based methods. Table 5 reports this cost on every workload, covering both operands of each query. The conversion costs at most 7.40% of the AMS-HYB runtime, with the two largest shares on IMDB and ISF18, which have the lowest $|OUT_J|/|tuple|$ ratios and therefore the least multiplication over which to amortize the linear pass. On the remaining eight workloads it stays below 1.2% under both query classes, so the input format is not a performance concern for the matrix-based methods.

Figure 19 presents the join-aggregate results. As shown, AMS-HYB consistently achieves the best performance among all aggregate variants, demonstrating its effectiveness in addressing relational join queries. Surprisingly, even compared with the join-only baseline NPHJ, AMS-HYB wins 9 out of 10 cases, losing only on BKGN, highlighting the superiority of SpGEMM methods in handling joins with large output cardinalities. Comparing the two runners-up, MKL and DIM3, DIM3 is the weaker of the two, trailing MKL by 1.65× in geometric mean over the workloads both complete, and it fails to finish on some workloads due to out-of-memory errors. This is because DIM3 uses an "estimate-partition-multiplication" approach that tends to over-estimate output size under skew, a common scenario in real-world workloads, and thus incurs significant memory materialization cost. Moreover, DIM3 and other prior matrix-based works [31, 51, 52, 101] mainly target an algorithmic remapping, i.e., expressing the query as a sparse matrix product, and leave the parallelization to a plain OpenMP loop without a scheduling design of their own. DIM3 therefore inherits exactly the load imbalance that AMS targets, which is why it generally trails MKL, an industrial standard with mature thread scalability, even where DIM3 completes. Notably, AMS-HYB outperforms MKL by 1.37× in geometric mean and by up to 3.69× on CLH0, a skewed workload with a sizable matrix of 850K rows and a Gini coefficient of 0.59. These results show that AMS's benefits from addressing multi-dimensional skew and reducing on-chip cache contention translate directly to skewed relational workloads. These gains are

[19] For join-project, only existence matters, so a recorded pair is the nonzero entry itself and the row-wise tuple can drop the value, leaving ⟨row-index, column-index⟩ alone as exactly the COO of a sparse Boolean matrix.

most pronounced on large, skewed workloads, where load imbalance and cache contention can make memory accesses dominate runtime. Smaller workloads, by contrast, fit largely in cache, leaving less memory-access overhead for AMS to reduce.

The hash-join baselines face more severe limitations on larger workloads: of the ten workloads, NPHJ-AGG completes only the three smallest, while PHJ-AGG completes only the two smallest. This is because, regardless of whether the aggregation is performed on-the-fly or after the join result is fully populated, the hash table must grow dynamically to hold the unhashed entries that are to be aggregated. This dynamically growing hash table is usually resized with a specific scaling factor (typically 2×), and therefore exhausts the memory footprint very quickly. Beyond memory limitations, comparing the "join-then-aggregate" approaches with their respective join-only baselines, we can see that the aggregation-based methods are more than an order of magnitude slower on average, indicating the significant cost of aggregation, and thus underscoring the necessity and importance of SpGEMM-based methods. This is particularly true for PHJ and PHJ-AGG: PHJ alone achieves the best runtime on all workloads but CLH0, yet aggregation inflates its runtime by more than two orders of magnitude wherever PHJ-AGG completes (for instance from 0.041 to 7.852 seconds on IMDB). This stark contrast indicates that the aggregation operation itself is the dominant bottleneck, underscoring the need for effective join-aggregate query processing.

We now turn to the join-project setting, which shares the same joins, datasets, and skew. Figure 20 reports this second setting, and the performance ranking of the methods carries over unchanged. AMS-HYB again achieves the best performance among all join-project variants. MKL also follows as the closest competitor, trailing by 1.33× in geometric mean and by up to 2.19× on MVLN. DIM3 again ranks third, but its gap behind AMS-HYB widens from 2.28× in join-aggregate to 2.64× in join-project, measured in geometric mean over the workloads it completes. It also continues to fail on the same four workloads with the largest outputs, for the reasons discussed previously.

Compared with the join-aggregate results, the join-project workloads generally exhibit lower runtime, as the query no longer requires the value computation and thereby incurs less arithmetic cost and less memory traffic. AMS-HYB, DIM3, and MKL accordingly run 1.66×, 1.34×, and 1.71× faster in geometric mean than their respective join-aggregate counterparts. DIM3, however, still falls behind MKL by 2.10× in geometric mean over the workloads both complete, losing on five of the six, even though DIM3 uses early termination and MKL does not. This underscores that, without proper scheduling and load balancing, performing less work does not always translate into higher performance. AMS-HYB, in contrast, pairs early termination with contention-aware scheduling and load balancing, and therefore converts the reduced work into an actual runtime saving.

The hash-join baselines face similar limitations to their join-aggregate counterparts, with NPHJ-PRJ and PHJ-PRJ each completing only three of the ten workloads. This is because both require a dynamically growing hash table for projection, whether it is performed on-the-fly or after the join result is fully materialized. As the table grows, it typically doubles in size with each resize, eventually exhausting memory on larger workloads. Beyond these memory limitations, NPHJ-PRJ remains up to 17× slower than the join-only NPHJ on workloads it completes, showing that projection dominates the runtime. AMS-HYB again wins 9 of the 10 cases against NPHJ itself, losing only on BKGN, which reaffirms the superiority of SpGEMM-based methods on relational join-project queries.

Table 6: M_l construction on DBLP V10 (seconds).

Meta-path	$\|row\|$	$\|nnz\|$	Gini	AMS-HYB	MKL	Speedup
APCPA	1.77M	4.98M	0.62	0.052	0.117	2.24×
CPAPC	5.08K	4.98M	0.78	0.067	0.120	1.80×
APTPA	1.77M	48.48M	0.60	0.158	0.529	3.34×
TPAPT	120.69K	48.48M	0.94	0.235	0.382	1.62×
Geomean	–	–	–	0.107	0.231	2.16×

4.7 Graph-Analytic Evaluation

Real-world graph workloads are typically sparse, and are commonly modeled as sparse matrices through their adjacency representation, which places one nonzero at every edge, and in this representation, expanding a set of vertices to their neighbors can be expressed as a matrix product. SpGEMM is therefore an important building block of graph analytics, expressing operators such as frontier expansion, neighborhood composition, and masked edge enumeration. Moreover, many real-world graphs have power-law degree distributions, which lead to skewed row lengths in their adjacency matrices. This makes them well suited to AMS's contention-aware scheduling and load balancing. We evaluate three representative graph-analytic workloads, namely meta-path construction for PathSim similarity search, breadth-first search, and triangle counting.

4.7.1 PathSim Meta-Path Construction. We first evaluate AMS on PathSim [103], a meta-path-based similarity measure for heterogeneous information networks, focusing on the intermediate matrices it must construct. Given a symmetric meta-path, PathSim scores node pairs through a commuting matrix $M = M_l M_l^T$, whose left half M_l is a chain of sparse matrix products. Constructing M_l is therefore an SpGEMM workload, and it dominates the cost of answering a query.

We evaluate on the AMiner DBLP V10 citation network [106], comprising 1.77M authors (A), 3.08M papers (P), 5,078 venues (C), and 120,685 terms (T), with 9.48M author-paper, 2.57M venue-paper, and 22.64M paper-term edges. We select four symmetric meta-paths that place each node type in the row dimension: APCPA and APTPA relate authors through shared venues and terms, whereas TPAPT and CPAPC relate terms and venues through shared authors. As Table 6 summarizes, they differ widely in both shape and skew, and form two transpose pairs (APCPA/CPAPC and APTPA/TPAPT) that perform the same work over row counts differing by 350× and 15×. All four exhibit pronounced row-wise skew (Gini 0.60–0.94), reflecting prevalent skew in real-world graphs. We compare AMS-HYB against the industry-standard MKL and report the SpGEMM time of the M_l construction.

As reported in Table 6, AMS-HYB outperforms MKL on all four meta-paths, by 1.62× to 3.34×, giving a geometric-mean speedup of 2.16×. These gains span a 350× range of row counts, from the 5,078-row CPAPC to the 1.77M-row APTPA, indicating that the benefit

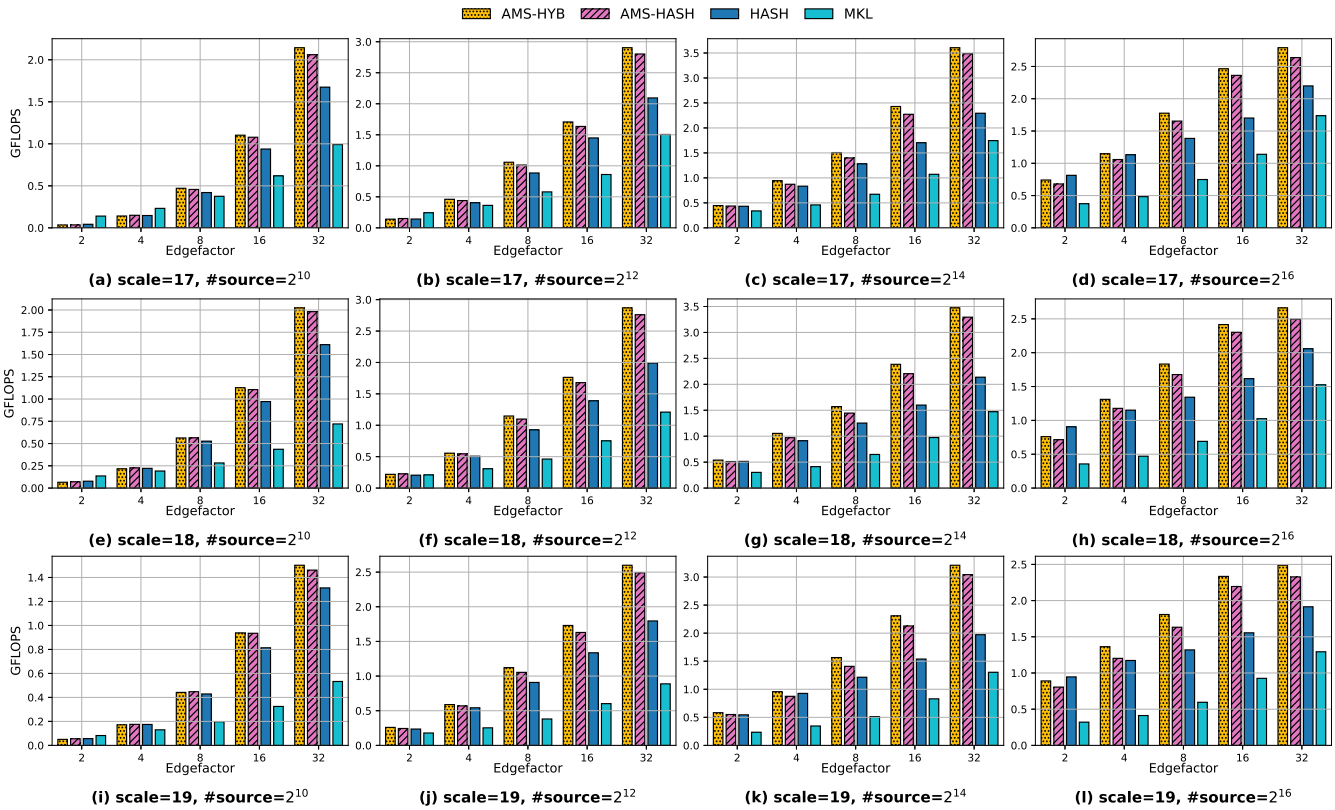


Figure 21: Multi-source BFS throughput on synthetic G500 graphs, where a frontier matrix holding #*source* sources is repeatedly multiplied by the adjacency matrix. Subfigures (a)–(l) sweep the graph *scale* over 17–19 and the number of sources over 2^{10}–2^{16}, with *edgefactor* varying along the x-axis of every panel.

of AMS does not depend on a particular matrix shape. Within each transpose pair, however, the gain is consistently larger for the variant with more rows (2.24× against 1.80×, and 3.34× against 1.62×), as the greater computational work associated with the larger row count helps amortize the cost of cf-based morsel sorting. Overall, AMS extends its advantage beyond matrix squaring and relational queries to meta-path construction on heterogeneous networks.

4.7.2 Breadth-First Search. We next evaluate AMS on multi-source breadth-first search (BFS), a traversal primitive used in many graph-analytic tasks [32]. For example, betweenness centrality measures how often a vertex lies on shortest paths between other vertices. Computing it on an unweighted graph requires a BFS from each source vertex to obtain shortest-path counts, making repeated traversals over the same graph a major part of its cost. Beyond centrality, BFS also supports connected components, reachability and k-hop neighborhood queries, unweighted single-source shortest paths, and diameter or eccentricity estimation. Faster traversal can therefore accelerate a broad range of graph-analytic workloads.

SpGEMM is exactly the kernel that carries out one traversal level in the linear-algebraic formulation [84]. The left operand is the graph's square adjacency matrix, while the right operand is a tall-skinny frontier matrix with one column per source, marking the vertices reached at the current level. Following the row-wise formulation in § 2.1, each adjacency row selects and accumulates the frontier rows corresponding to its neighbors. The vertex degree therefore determines how many frontier rows must be processed, while a single matrix product advances all sources by one hop. Multi-source BFS thus consists of repeated square-by-tall-skinny products, whose row-wise work reflects the graph's degree distribution. Since real-world graphs commonly exhibit power-law degree distributions, these products naturally exhibit the row-wise skew that AMS is designed to handle.

Following the setup of Nagasaka et al. [84], we evaluate the BFS kernel as the product of a square graph adjacency matrix and a tall-skinny frontier matrix on synthetic G500 graphs. These graphs capture the power-law degree distributions common in real-world graphs while allowing the frontier width to vary independently of graph size and density. We sweep the graph *scale* over 17–19 and the number of concurrent sources over 2^{10}–2^{16}, and extend that setup by sweeping the *edgefactor* over 2–32 as well, so that density varies alongside frontier width. As the frontier is used as a set of reached vertices, frontier expansion requires no output ordering, so we report only the non-sorted methods. We compare AMS-HYB and AMS-HASH against the HASH baseline and the non-sorted Intel MKL variant.

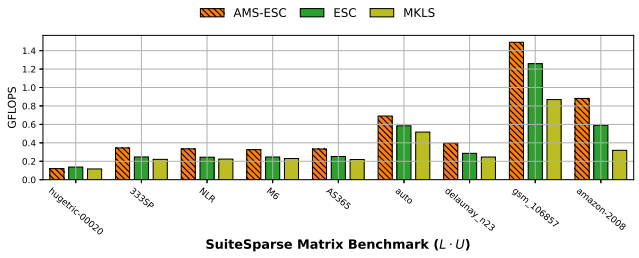


Figure 22: Triangle counting throughput on nine SuiteSparse matrices.

Figure 21 shows that AMS-HYB achieves the highest throughput among the four methods on 47 of the 60 evaluated workloads. It outperforms the HASH baseline on 50 workloads, with a geometric-mean speedup of 1.18× and a maximum speedup of 1.63×, while AMS-HASH achieves a geometric-mean speedup of 1.13× over the same baseline. The advantage over MKL is larger: AMS-HYB achieves a geometric-mean speedup of 1.91× and a maximum speedup of 3.31×.

The remaining 13 workloads, where AMS-HYB does not achieve the highest throughput, occur at the extremes of the parameter sweep. MKL wins on 5 workloads with low density and few sources (*edgefactor* 2–4 and 2^{10}–2^{12} sources), where the limited computation is insufficient to amortize AMS's morsel preparation cost. The HASH baseline wins on another 3, all at *edgefactor* 2 with the widest frontier (2^{16} sources). Finally, AMS-HASH wins on the remaining 5 by margins below 5%, with 4 of these occurring at 2^{10} sources.

From Figure 21, we can observe two trends in AMS-HYB's speedup over the HASH baseline. First, the speedup increases steadily with graph density, from 0.96× at *edgefactor* 2 to 1.38× at *edgefactor* 32. Only at the lowest density does AMS-HYB fall behind the baseline, because the computation is insufficient to amortize the cost of cf-based morsel sorting, consistent with the matrix-squaring results in § 4.2. Second, the speedup increases with the number of concurrent sources, from 1.05× at 2^{10} sources to 1.20× at 2^{16} sources. A wider frontier increases the work performed at each traversal level, creating more cache contention for CALB to alleviate. Together, these trends suggest that AMS benefits most when the workload provides enough computation to amortize scheduling overhead and enough skew-induced contention for its scheduling strategy to address.

4.7.3 Triangle Counting. Finally, we evaluate AMS on triangle counting [105], which counts the triangles in a graph. This operation supports clustering coefficient and transitivity calculations, as well as community detection and spam or anomaly detection in web and social networks. Triangle counting typically operates on an undirected graph, represented by a square, symmetric sparse adjacency matrix. In the formulation we evaluate, multiplying its strictly lower and upper triangular parts, $L \cdot U$, identifies paths of length two, also called wedges. The product is then masked with the adjacency matrix to retain only wedges whose endpoints are connected by an edge, forming a triangle. To apply this mask, each output row is merged with the corresponding adjacency row, requiring the entries within each output row to be sorted.

Following the setup of Nagasaka et al. [84], we evaluate triangle counting on the nine square, structurally symmetric matrices in Table 1. For each matrix, we first reorder the vertices by increasing row nonzero count, applying the same permutation to rows and columns to preserve symmetry. We then split the reordered matrix into its strictly lower and upper triangular parts, L and U, to form the SpGEMM operands. Since the subsequent masking step requires row-wise sorted output, we compare only sorted methods: AMS-ESC, the ESC baseline, and the sorted Intel MKL variant (MKLS).

Figure 22 reports the results. AMS-ESC outperforms the ESC baseline on 8 of the 9 matrices, achieving a geometric-mean speedup of 1.27× and a maximum speedup of 1.49×. It also outperforms MKLS on all 9 matrices, with a geometric-mean speedup of 1.55× and a maximum speedup of 2.76×. The largest gains over MKLS occur on the two most skewed matrices, amazon-2008 and gsm_106857, with speedups of 2.76× and 1.72×, respectively. The only loss to the ESC baseline occurs on hugetric-00020, the least skewed matrix in the benchmark (Gini 0.0003), where there is little load imbalance to alleviate and insufficient computation to amortize the cost of cf-based morsel sorting. Together, these results demonstrate that AMS's benefits extend to real-world graph workloads requiring sorted output, with the greatest gains on skewed matrices.

4.8 Discussion

Our experimental analysis across sparse matrix multiplication, relational workloads, and graph analytics, on diverse datasets and hardware platforms, reveals several key findings that collectively validate the design rationale and practical effectiveness of AMS.

Multi-dimensional skew is the primary performance bottleneck. Across all matrix squaring evaluations (§ 4.2), we observe that the performance gap between AMS and flop-count-based baselines widens as workload skew increases. On non-skewed ER matrices, AMS performs comparably to the baselines, confirming that it introduces negligible overhead. On skewed G500 matrices, however, AMS delivers substantial gains (up to 81.01% for HASH), demonstrating that multi-dimensional skew, not algorithmic inefficiency, is the dominant factor limiting existing methods. That said, AMS may occasionally underperform in highly sparse cases due to overhead from cf-based morsel sorting; however, this penalty diminishes rapidly as matrix density increases, and is outweighed by the substantial gains on the skewed workloads that are central to real-world applications.

Contention-aware scheduling is essential, not optional. The dissection of CALB and SMTact (§ 4.3) shows that simply achieving load balance is insufficient: methods that balance load but ignore cache contention (e.g., OpenMP dynamic and guided scheduling) still fall short of CALB. Similarly, SMTact offers a robust default that avoids the substantial degradation that either fixed policy incurs outside its sweet spot, confirming that the trade-off between computational throughput and cache pressure must be managed explicitly rather than left to a fixed policy.

Load balancing unlocks hardware potential. The CXL evaluation (§ 4.4) demonstrates that load imbalance can negate the benefits of bandwidth expansion, as straggler threads are disproportionately penalized by CXL's higher access latency. By addressing this

imbalance, AMS enables effective utilization of the augmented bandwidth, achieving throughput that exceeds the DRAM-only peak. This finding has broader implications. As heterogeneous memory systems become more prevalent [62, 63, 77, 78, 117], contention-aware load balancing will be increasingly critical for realizing their performance potential.

The framework generalizes to other sparse kernels. The SpMM and SpMV evaluation (§ 4.5) shows that AMS's scheduling principles extend beyond SpGEMM to other sparse kernels with irregular sparsity patterns. The bandwidth measurements help explain these gains: on skewed workloads, the baseline uses only a fraction of the available DRAM bandwidth, while AMS achieves higher bandwidth utilization. Since both kernels are memory-bound, contention-aware scheduling improves memory bandwidth utilization and thereby increases throughput.

SpGEMM-based methods are superior for relational query processing. The join-aggregate and join-project evaluation (§ 4.6) demonstrates the advantages of matrix-based methods, particularly when the intermediate join result is large. Traditional join-then-aggregate/project methods struggle with the resulting memory and processing costs, completing only the smallest workloads. SpGEMM-based methods avoid materializing this intermediate result and therefore run substantially faster. However, expressing these queries as sparse matrix products is only part of the solution. The performance limitations of prior matrix-based methods [31, 51, 52, 101] highlight the need for efficient parallel execution. AMS addresses this need through a load-balancing strategy that accounts for cache contention, translating the algorithmic benefits of the matrix formulation into consistent runtime gains.

The gains extend across graph-analytic workloads. These benefits also extend to graph analytics, as shown by the evaluation in § 4.7. AMS outperforms the strongest baseline across PathSim meta-path construction, multi-source BFS with non-sorted output, and triangle counting with sorted output. Together, these workloads demonstrate its effectiveness across different operand shapes, including tall-skinny and triangular matrices, and with both sorted and non-sorted output. Across these workloads, the gains are generally larger on more skewed inputs, reinforcing the practical value of contention-aware scheduling under skew.

5 Related Work

SpGEMM has been extensively studied as a critical kernel in scientific computing and graph analytics [29, 87], and more recently in relational query processing [31, 51, 52, 58, 67, 101]. These relational works, however, are largely confined to the algorithmic mapping from queries to sparse matrix products, and address neither the efficient parallel execution of the resulting kernel nor the load imbalance under skew that AMS targets. Among them, SPALM [67] is the most closely related to our work: it partitions the operands into small slices and selects a multiplication kernel for each slice pair based on local density and machine-calibrated thresholds. Its adaptivity focuses on kernel selection, leaving load balancing and cache contention outside its scope. AMS, in contrast, adapts the size and execution order of parallel tasks to address both issues, making the two approaches complementary.

Beneath every such mapping lies the SpGEMM kernel itself, whose optimization has been studied extensively. In this line of work, Gustavson's row-row formulation [48] has over the decades become the dominant approach due to its natural parallelizability, particularly when paired with the dense array accumulator known as SPA [42]. However, SPA has been shown to be inefficient and to incur high memory overhead, which prompted the development of more memory-efficient alternatives such as hash tables [84, 92], heaps [83], and radix sort [29].

In addition to algorithmic advances, much attention has been given to the challenges of load balancing in parallel SpGEMM. Schemes based on flop counts [83, 84] have emerged as a dominant solution, along with other static strategies such as methods using blocks [18, 19] or graph partitioning [2]. While effective in certain scenarios, these approaches often fail to account for the multi-dimensional skew characteristics present in real-world SpGEMM workloads, resulting in imbalanced execution.

As a more adaptive alternative, dynamic scheduling offers greater flexibility. For instance, Patwary et al. [88] proposed a dynamic task scheduling scheme that partitions work across both row and column dimensions. However, task partitioning across two dimensions introduces substantial synchronization overhead, which limits its practical applicability. Similarly, techniques leveraging work-stealing [14, 15, 123] and cache-access modeling [47] have been explored, but either incur significant cache coherence overhead, or fail to account for complicated LLC sharing characteristics, making them less favorable than the flop-count-based approaches. Alongside these approaches, researchers have adapted dynamic variants of the classical longest-processing-time-first (LPT) algorithm [44], which schedules heavier tasks before lighter ones. However, none of these approaches account for on-chip cache contention caused by processing dense submatrices concurrently, an issue that AMS explicitly addresses alongside multi-dimensional skew.

Prior work has also explored task granularity in SpGEMM through one- and two-dimensional blocking [18, 19], sparse tiles [49, 86], and adaptive panels [122]. These approaches balance spatial locality against the flexibility to distribute work across threads. A similar trade-off arises in data systems, where morsel-driven parallelism [33, 73] distributes fixed-size groups of *input* tuples across workers. AMS adopts the morsel abstraction but varies the input size to maintain a roughly fixed *output* volume per morsel. It also orders morsel execution to reduce shared-cache contention, which conventional morsel-driven parallelism does not explicitly address. The approaches are therefore complementary, and AMS's techniques could benefit morsel-driven engines by using partition-size estimates to guide dispatch order and SMT activation for hash-based operators under skew.

6 Conclusion

This paper presents adaptive morsel scheduling (AMS), a morsel-wise dynamic scheduling framework that addresses two co-existing challenges in SpGEMM, namely *multi-dimensional skew* and *on-chip cache contention*. AMS balances skewed workloads without aggravating cache contention through contention-aware load balancing (CALB), judiciously activates SMT via data-conscious simultaneous multithreading activator (SMTact), and adaptively selects

the per-morsel accumulator with hybrid accumulation mechanism (HYB). Evaluations across matrix squaring, relational queries, graph analytics, and other sparse kernels show that AMS significantly outperforms state-of-the-art methods on skewed workloads while maintaining comparable performance on non-skewed workloads. It also improves memory bandwidth utilization, with these benefits observed on both conventional and CXL-expanded memory systems. Looking ahead, extending the morsel-wise scheduling framework to other SpGEMM formulations, such as outer-product and inner-product methods, could further improve performance on highly sparse matrices.

References

[1] Christopher R. Aberger, Andrew Lamb, Kunle Olukotun, and Christopher Ré. 2018. LevelHeaded: A Unified Engine for Business Intelligence and Linear Algebra Querying. In *ICDE*. IEEE Computer Society, 449–460.

[2] Kadir Akbudak and Cevdet Aykanat. 2014. Simultaneous Input and Output Matrix Partitioning for Outer-Product-Parallel Sparse Matrix-Matrix Multiplication. *SIAM J. Sci. Comput.* 36, 5 (2014).

[3] Rasmus Resen Amossen and Rasmus Pagh. 2009. Faster join-projects and sparse matrix multiplications. In *Proceedings of the 12th International Conference on Database Theory*. 121–126.

[4] Junya Arai, Hiroaki Shiokawa, Takeshi Yamamuro, Makoto Onizuka, and Sotetsu Iwamura. 2016. Rabbit Order: Just-in-Time Parallel Reordering for Fast Graph Analysis. In *IPDPS*. IEEE Computer Society, 22–31.

[5] Ariful Azad, Grey Ballard, Aydin Buluç, James Demmel, Laura Grigori, Oded Schwartz, Sivan Toledo, and Samuel Williams. 2016. Exploiting Multiple Levels of Parallelism in Sparse Matrix-Matrix Multiplication. *SIAM J. Sci. Comput.* 38, 6 (2016).

[6] Cagri Balkesen, Gustavo Alonso, Jens Teubner, and M. Tamer Özsu. 2013. Multi-Core, Main-Memory Joins: Sort vs. Hash Revisited. *Proc. VLDB Endow.* 7, 1 (2013), 85–96.

[7] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2013. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware. In *ICDE*. IEEE Computer Society, 362–373.

[8] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2015. Main-Memory Hash Joins on Modern Processor Architectures. *IEEE Trans. Knowl. Data Eng.* 27, 7 (2015), 1754–1766.

[9] Grey Ballard, Alex Druinsky, Nicholas Knight, and Oded Schwartz. 2016. Hypergraph Partitioning for Sparse Matrix-Matrix Multiplication. *ACM Trans. Parallel Comput.* 3, 3 (2016), 18:1–18:34.

[10] Maximilian Bandle, Jana Giceva, and Thomas Neumann. 2021. To Partition, or Not to Partition, That is the Join Question in a Real System. In *SIGMOD Conference*. ACM, 168–180.

[11] Richard Barrett, Michael Berry, Tony F Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk Van der Vorst. 1994. *Templates for the solution of linear systems: building blocks for iterative methods*. SIAM.

[12] Nathan Bell, Steven Dalton, and Luke N Olson. 2012. Exposing fine-grained parallelism in algebraic multigrid methods. *SIAM Journal on Scientific Computing* 34, 4 (2012), C123–C152.

[13] Spyros Blanas, Yinan Li, and Jignesh M. Patel. 2011. Design and evaluation of main memory hash join algorithms for multi-core CPUs. In *SIGMOD Conference*. ACM, 37–48.

[14] Robert D. Blumofe. 1994. Scheduling Multithreaded Computations by Work Stealing. In *FOCS*. IEEE Computer Society, 356–368.

[15] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. 1995. Cilk: An Efficient Multithreaded Runtime System. In *PPoPP*. ACM, 207–216.

[16] OpenMP Architecture Review Board. 1997. OpenMP Fortran Application Program Interface. (1997).

[17] Peter A. Boncz, Stefan Manegold, and Martin L. Kersten. 1999. Database Architecture Optimized for the New Bottleneck: Memory Access. In *VLDB*. Morgan Kaufmann, 54–65.

[18] Aydin Buluç and John R. Gilbert. 2008. Challenges and Advances in Parallel Sparse Matrix-Matrix Multiplication. In *ICPP*. IEEE Computer Society, 503–510.

[19] Aydin Buluç and John R. Gilbert. 2008. On the representation and multiplication of hypersparse matrices. In *IPDPS*. IEEE, 1–11.

[20] Aydin Buluç and John R. Gilbert. 2011. The Combinatorial BLAS: design, implementation, and applications. *Int. J. High Perform. Comput. Appl.* 25, 4 (2011), 496–509.

[21] Aydin Buluç and John R. Gilbert. 2012. Parallel Sparse Matrix-Matrix Multiplication and Indexing: Implementation and Experiments. *SIAM J. Sci. Comput.* 34, 4 (2012).

[22] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A Recursive Model for Graph Mining. In *SDM*. SIAM, 442–446.

[23] Surajit Chaudhuri and Kyuseok Shim. 1994. Including Group-By in Query Optimization. In *VLDB'94, Proceedings of 20th International Conference on Very Large Data Bases, September 12-15, 1994, Santiago de Chile, Chile*, Jorge B. Bocca, Matthias Jarke, and Carlo Zaniolo (Eds.). Morgan Kaufmann, 354–366.

[24] YuAng Chen and Yeh-Ching Chung. 2022. Workload Balancing via Graph Reordering on Multicore Systems. *IEEE Trans. Parallel Distributed Syst.* 33, 5 (2022), 1231–1245.

[25] Yu Chen and Ke Yi. 2017. Two-Level Sampling for Join Size Estimation. In *SIGMOD Conference*. ACM, 759–774.

[26] Helin Cheng, Wenxuan Li, Yuechen Lu, and Weifeng Liu. 2023. HASpGEMM: Heterogeneity-Aware Sparse General Matrix-Matrix Multiplication on Modern Asymmetric Multicore Processors. In *ICPP*. ACM, 807–817.

[27] Transaction Processing Performance Council. 2021. TPC BENCHMARKTM DS Standard Specification Revision 3.2.0. https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-ds_v3.2.0.pdf Accessed: October 2026.

[28] Leonardo Dagum and Ramesh Menon. 1998. OpenMP: an industry standard API for shared-memory programming. *IEEE computational science and engineering* 5, 1 (1998), 46–55.

[29] Steven Dalton, Luke Olson, and Nathan Bell. 2015. Optimizing sparse matrix—matrix multiplication for the gpu. *ACM Transactions on Mathematical Software (TOMS)* 41, 4 (2015), 1–20.

[30] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38, 1 (2011), 1:1–1:25.

[31] Shaleen Deep, Xiao Hu, and Paraschos Koutris. 2020. Fast join project query evaluation using matrix multiplication. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1213–1223.

[32] Laxman Dhulipala, Charles McGuffey, Hongbo Kang, Yan Gu, Guy E. Blelloch, Phillip B. Gibbons, and Julian Shun. 2020. Sage: Parallel Semi-Asymmetric Graph Algorithms for NVRAMs. *Proc. VLDB Endow.* 13, 9 (2020), 1598–1613.

[33] Kayhan Dursun, Carsten Binnig, Ugur Çetintemel, Garret Swart, and Weiwei Gong. 2019. A Morsel-Driven Query Execution Engine for Heterogeneous Multi-Cores. *Proc. VLDB Endow.* 12, 12 (2019), 2218–2229.

[34] Marius Eich, Pit Fender, and Guido Moerkotte. 2018. Efficient generation of query plans containing group-by, join, and groupjoin. *VLDB J.* 27, 5 (2018), 617–641.

[35] Philipp Fent, Altan Birler, and Thomas Neumann. 2023. Practical planning and execution of groupjoin and nested aggregates. *VLDB J.* 32, 6 (2023), 1165–1190.

[36] Philipp Fent and Thomas Neumann. 2021. A Practical Approach to Groupjoin and Nested Aggregates. *Proc. VLDB Endow.* 14, 11 (2021), 2383–2396.

[37] Patrick J. Flynn, Xinyao Yi, Erik Saule, Gokcen Kestor, and Yonghong Yan. 2025. SpMM-Bench: Performance Characterization of Sparse Formats for Sparse-Dense Matrix Multiplication. In *IPDPS (Workshops)*. IEEE, 149–158.

[38] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. 2023. A Systematic Survey of General Sparse Matrix-matrix Multiplication. *ACM Comput. Surv.* 55, 12 (2023), 244:1–244:36.

[39] Jianhua Gao, Bingjie Liu, Weixing Ji, and Hua Huang. 2024. A Systematic Literature Survey of Sparse Matrix-Vector Multiplication. *CoRR* abs/2404.06047 (2024).

[40] Sen Gao, Shengliang Lu, Shixuan Sun, Yuchen Li, and Bingsheng He. 2025. An Efficient Memoization Engine for Concurrent Graph Query Processing. In *ICDE*. IEEE, 1015–1028.

[41] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39.

[42] John R Gilbert, Cleve Moler, and Robert Schreiber. 1992. Sparse matrices in MATLAB: Design and implementation. *SIAM journal on matrix analysis and applications* 13, 1 (1992), 333–356.

[43] John R. Gilbert, Steven P. Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *PARA (Lecture Notes in Computer Science)*, Vol. 4699. Springer, 260–269.

[44] Ronald L. Graham. 1969. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal of Applied Mathematics* 17, 2 (1969), 416–429.

[45] Zhixiang Gu, Jose Moreira, David Edelsohn, and Ariful Azad. 2020. Bandwidth Optimized Parallel Algorithms for Sparse Matrix-Matrix Multiplication using Propagation Blocking. In *SPAA*. ACM, 293–303.

[46] Gaël Guennebaud, Benoît Jacob, et al. 2010. Eigen. https://libeigen.gitlab.io/ Accessed: October 2026.

[47] Jihu Guo, Rui Xia, Jie Liu, Xiaoxiong Zhu, and Xiang Zhang. 2024. CAMLB-SpMV: An Efficient Cache-Aware Memory Load-Balancing SpMV on CPU. In *ICPP*. ACM, 640–649.

[48] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (1978), 250–269.

[49] Changwan Hong, Aravind Sukumaran-Rajam, Israt Nisa, Kunal Singh, and P. Sadayappan. 2019. Adaptive sparse tiling for sparse matrix multiplication. In

PPoPP. ACM, 300–314.

[50] Chuhe Hong, Qinglin Wang, Runzhang Mao, Yuechao Liang, Rui Xia, and Jie Liu. 2024. SaSpGEMM: Sorting-Avoiding Sparse General Matrix-Matrix Multiplication on Multi-Core Processors. In *ICPP*. ACM, 1166–1175.

[51] Xiao Hu. 2025. Output-optimal algorithms for join-aggregate queries. *Proceedings of the ACM on Management of Data* 3, 2 (2025), 1–27.

[52] Xiao Hu and Ke Yi. 2020. Parallel algorithms for sparse matrix multiplication and join-aggregate queries. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 411–425.

[53] Yu-Ching Hu, Yuliang Li, and Hung-Wei Tseng. 2022. TCUDB: Accelerating Database with Tensor Processors. In *SIGMOD Conference*. ACM, 1360–1374.

[54] Wentao Huang, Yunhong Ji, Xuan Zhou, Bingsheng He, and Kian-Lee Tan. 2023. A Design Space Exploration and Evaluation for Main-Memory Hash Joins in Storage Class Memory. *Proc. VLDB Endow.* 16, 6 (2023), 1249–1263.

[55] Wentao Huang, Yuchen Li, Yuan Fang, Ju Fan, and Hongxia Yang. 2020. BiANE: Bipartite Attributed Network Embedding. In *SIGIR*. ACM, 149–158.

[56] Wentao Huang, Mian Lu, and Kian-Lee Tan. 2026. Hash Joins Meet CXL: A Fresh Look. In *CIDR*. www.cidrdb.org.

[57] Wentao Huang, Mo Sha, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. 2024. Bandwidth Expansion via CXL: A Pathway to Accelerating In-Memory Analytical Processing. In *VLDB Workshops*. VLDB.org.

[58] Zichun Huang and Shimin Chen. 2022. Density-optimized intersection-free mapping and matrix multiplication for join-project operations. *Proceedings of the VLDB Endowment* 15, 10 (2022), 2244–2256.

[59] IMDb.com, Inc. 2024. IMDb Non-Commercial Datasets. https://developer.imdb.com/non-commercial-datasets/ Accessed: October 2026.

[60] Intel. 2024. Intel® oneAPI Math Kernel Library (oneMKL). https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html Accessed: October 2026.

[61] Myung-Hwan Jang, Yunyong Ko, Hyuck-Moo Gwon, Ikhyeon Jo, Yongjun Park, and Sang-Wook Kim. 2023. SAGE: a storage-based approach for scalable and efficient sparse generalized matrix-matrix multiplication. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 923–933.

[62] Yunhong Ji, Wentao Huang, and Xuan Zhou. 2024. HeterMM: applying in-DRAM index to heterogeneous memory-based key-value stores. *Frontiers Comput. Sci.* 18, 4 (2024), 184612.

[63] Yunhong Ji, Wentao Huang, Xuan Zhou, Bingsheng He, and Kian-Lee Tan. 2024. TaC: An Anti-Caching Key-Value Store on Heterogeneous Memory Architectures. In *EDBT*. OpenProceedings.org, 474–487.

[64] Mahmoud Abo Khamis, Xiao Hu, and Dan Suciu. 2025. Fast Matrix Multiplication meets the Submodular Width. *Proc. ACM Manag. Data* 3, 2 (2025), 98:1–98:26.

[65] Farzad Khorasani, Rajiv Gupta, and Laxmi N. Bhuyan. 2015. Scalable SIMD-Efficient Graph Processing on GPUs. In *Proceedings of the 24th International Conference on Parallel Architectures and Compilation Techniques (PACT '15)*. 39–50.

[66] Changkyu Kim, Eric Sedlar, Jatin Chhugani, Tim Kaldewey, Anthony D. Nguyen, Andrea Di Blas, Victor W. Lee, Nadathur Satish, and Pradeep Dubey. 2009. Sort vs. Hash Revisited: Fast Join Implementation on Modern Multi-Core CPUs. *Proc. VLDB Endow.* 2, 2 (2009), 1378–1389.

[67] Junyoung Kim and Kenneth A. Ross. 2026. SPALM: A Sparsity-Pattern-Adaptive Library for Matrices. *Proc. ACM Manag. Data* 4, 3 (2026), 236:1–236:27.

[68] Denis Kotkov, Joseph A. Konstan, Qian Zhao, and Jari Veijalainen. 2018. Investigating serendipity in recommender systems based on real user feedback. In *SAC*. ACM, 1341–1350.

[69] Denis Kotkov, Alan Medlar, Alexandr Maslov, Umesh Raj Satyal, Mats Neovius, and Dorota Glowacka. 2022. The Tag Genome Dataset for Books. In *Proceedings of the 2022 ACM SIGIR Conference on Human Information Interaction and Retrieval (CHIIR '22)*. 5. https://doi.org/10.1145/3498366.3505833

[70] Jérôme Kunegis and Julia Preusse. 2012. Fairness on the web: alternatives to the power law. In *WebSci*. ACM, 175–184.

[71] Georgiy Lebedev, Hamish Nicholson, Musa Ünal, Sanidhya Kashyap, and Anastasia Ailamaki. 2025. Demystifying CXL Memory Bandwidth Expansion for Analytical Workloads. In *VLDB Workshops*. VLDB.org.

[72] Suhyun Lee, Chaemin Lim, Jinwoo Choi, Heelim Choi, Chan Lee, Yongjun Park, Kwanghyun Park, Hanjun Kim, and Youngsok Kim. 2024. SPID-Join: A Skew-resistant Processing-in-DIMM Join Algorithm Exploiting the Bank- and Rank-level Parallelisms of DIMMs. *Proc. ACM Manag. Data* 2, 6 (2024), 251:1–251:27.

[73] Viktor Leis, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2014. Morsel-driven parallelism: a NUMA-aware query evaluation framework for the many-core age. In *SIGMOD Conference*. ACM, 743–754.

[74] Alberto Lerner and Gustavo Alonso. 2024. CXL and the Return of Scale-Up Database Engines. *Proc. VLDB Endow.* 17, 10 (2024), 2568–2575.

[75] Zhonggen Li, Xiangyu Ke, Yifan Zhu, Yunjun Gao, and Yaofeng Tu. 2025. HC-SpMM: Accelerating Sparse Matrix-Matrix Multiplication for Graphs with Hybrid GPU Cores. In *ICDE*. IEEE, 501–514.

[76] Chunxu Lin, Wensheng Luo, Yixiang Fang, Chenhao Ma, Xilin Liu, and Yuchi Ma. 2024. On Efficient Large Sparse Matrix Chain Multiplication. *Proc. ACM Manag. Data* 2, 3 (2024), 156.

[77] Jinshu Liu, Hamid Hadian, Yuyue Wang, Daniel S. Berger, Marie Nguyen, Xun Jian, Sam H. Noh, and Huaicheng Li. 2025. Systematic CXL Memory Characterization and Performance Analysis at Scale. In *ASPLOS (2)*. ACM, 1203–1217.

[78] Jinshu Liu, Hanchen Xu, Daniel S. Berger, Marcos K. Aguilera, and Huaicheng Li. 2026. Performance Predictability in Heterogeneous Memory. In *ASPLOS (2)*. ACM, 1413–1429.

[79] Shangyu Luo, Zekai J. Gao, Michael N. Gubanov, Luis Leopoldo Perez, and Christopher M. Jermaine. 2017. Scalable Linear Algebra on a Relational Database System. In *ICDE*. IEEE Computer Society, 523–534.

[80] Stefan Manegold, Peter A. Boncz, and Martin L. Kersten. 2002. Optimizing Main-Memory Join on Modern Hardware. *IEEE Trans. Knowl. Data Eng.* 14, 4 (2002), 709–730.

[81] Guido Moerkotte and Thomas Neumann. 2011. Accelerating Queries with Group-By and Join by Groupjoin. *Proc. VLDB Endow.* 4, 11 (2011), 843–851.

[82] Ingo Müller, Peter Sanders, Arnaud Lacurie, Wolfgang Lehner, and Franz Färber. 2015. Cache-Efficient Aggregation: Hashing Is Sorting. In *SIGMOD Conference*. ACM, 1123–1136.

[83] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydin Buluç. 2018. High-Performance Sparse Matrix-Matrix Products on Intel KNL and Multicore Architectures. In *ICPP Workshops*. ACM, 34:1–34:10.

[84] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydin Buluç. 2019. Performance optimization, modeling and analysis of sparse matrix-matrix products on multi-core and many-core processors. *Parallel Comput.* 90 (2019).

[85] Tien T. Nguyen, F. Maxwell Harper, Loren Terveen, and Joseph A. Konstan. 2018. User Personality and User Satisfaction with Recommender Systems. *Inf. Syst. Frontiers* 20, 6 (2018), 1173–1189.

[86] Yuyao Niu, Zhengyang Lu, Haonan Ji, Shuhui Song, Zhou Jin, and Weifeng Liu. 2022. TileSpGEMM: a tiled algorithm for parallel sparse general matrix-matrix multiplication on GPUs. In *PPoPP*. ACM, 90–106.

[87] Taehyeong Park, Seokwon Kang, Myung-Hwan Jang, Sang-Wook Kim, and Yongjun Park. 2023. Orchestrating Large-Scale SpGEMMs using Dynamic Block Distribution and Data Transfer Minimization on Heterogeneous Systems. In *ICDE*. IEEE, 2456–2459.

[88] Md. Mostofa Ali Patwary, Nadathur Rajagopalan Satish, Narayanan Sundaram, Jongsoo Park, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, Sergey G. Pudov, Vadim O. Pirogov, and Pradeep Dubey. 2015. Parallel Efficient Sparse Matrix-Matrix Multiplication on Multicore Platforms. In *ISC (Lecture Notes in Computer Science)*, Vol. 9137. Springer, 48–57.

[89] Constantine D. Polychronopoulos and David J. Kuck. 1987. Guided Self-Scheduling: A Practical Scheduling Scheme for Parallel Supercomputers. *IEEE Trans. Computers* 36, 12 (1987), 1425–1439.

[90] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 58–70. https://doi.org/10.1109/HPCA47549.2020.00015

[91] Moinuddin K. Qureshi. 2014. Memory Scaling is Dead, Long Live Memory Scaling. https://hps.ece.utexas.edu/yale75/qureshi_slides.pdf Accessed: October 2026.

[92] Stefan Richter, Victor Alvarez, and Jens Dittrich. 2015. A seven-dimensional analysis of hashing methods and its implications on query processing. *Proceedings of the VLDB Endowment* 9, 3 (2015), 96–107.

[93] Yousef Saad. 2003. *Iterative methods for sparse linear systems*. SIAM.

[94] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQL-Storm: Taking Database Benchmarking into the LLM Era. *Proc. VLDB Endow.* 18, 11 (2025), 4144–4157.

[95] Stefan Schuh, Xiao Chen, and Jens Dittrich. 2016. An Experimental Comparison of Thirteen Relational Equi-Joins in Main Memory. In *SIGMOD Conference*. ACM, 1961–1976.

[96] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse - Lightning Fast Analytics for Everyone. *Proc. VLDB Endow.* 17, 12 (2024), 3731–3744.

[97] Malte Skarupke. 2017. I Wrote The Fastest Hashtable. Probably Dance. https://probablydance.com/2017/02/26/i-wrote-the-fastest-hashtable/ Accessed: October 2026.

[98] SuiteSparse Matrix Collection. 2018. VLSI/vas_stokes_1M. https://sparse.tamu.edu/VLSI/vas_stokes_1M Accessed: October 2026.

[99] Ruixuan Sun, Ruoyan Kong, Qiao Jin, and Joseph A. Konstan. 2023. Less Can Be More: Exploring Population Rating Dispositions with Partitioned Models in Recommender Systems. In *UMAP (Adjunct Publication)*. ACM, 291–295.

[100] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2023. Accelerating Machine Learning Queries with Linear Algebra Query Processing. In *SSDBM*. ACM, 13:1–13:12.

[101] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2023. An Empirical Performance Comparison between Matrix Multiplication Join and Hash Join on GPUs.

In *ICDEW*. IEEE, 184–190.

[102] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2025. Accelerating machine learning queries with linear algebra query processing. *Distributed Parallel Databases* 43, 1 (2025), 8.

[103] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S. Yu, and Tianyi Wu. 2011. PathSim: Meta Path-Based Top-K Similarity Search in Heterogeneous Information Networks. *Proc. VLDB Endow.* 4, 11 (2011), 992–1003.

[104] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *MICRO*. ACM, 105–121.

[105] Narayanan Sundaram, Nadathur Satish, Md. Mostofa Ali Patwary, Subramanya Dulloor, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, and Pradeep Dubey. 2015. GraphMat: High performance graph analytics made productive. *Proc. VLDB Endow.* 8, 11 (2015), 1214–1225.

[106] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2017. DBLP-Citation-Network V10. AMiner Citation Network Dataset, release 10. https://lfs.aminer.cn/lab-datasets/citation/dblp.v10.zip Landing page: https://www.aminer.org/citation. 1.85 GB archive, 4 JSON shards, 3,079,007 papers. Accessed: October 2026.

[107] Yupeng Tang, Ping Zhou, Wenhui Zhang, Henry Hu, Qirui Yang, Hao Xiang, Tongping Liu, Jiaxin Shan, Ruoyun Huang, Cheng Zhao, Cheng Chen, Hui Zhang, Fei Liu, Shuai Zhang, Xiaoning Ding, and Jianjun Chen. 2024. Exploring Performance and Cost Optimization with ASIC-Based CXL Memory. In *EuroSys*. ACM, 818–833.

[108] Micron Technology. 2023. Memory Scaling is Dead, Long Live Memory Scaling. https://sg.micron.com/content/dam/micron/global/public/products/white-paper/cxl-impact-dram-bit-growth-white-paper.pdf Accessed: October 2026.

[109] The Linux Kernel Development Community. 2024. *Intel(R) Speed Select Technology User Guide*. https://docs.kernel.org/admin-guide/pm/intel-speed-select.html Linux Kernel Documentation.

[110] James D. Trotter, Sinan Ekmekçibasi, Johannes Langguth, Tugba Torun, Emre Düzakin, Aleksandar Ilic, and Didem Unat. 2023. Bringing Order to Sparsity: A Sparse Matrix Reordering Study on Multicore CPUs. In *SC*. ACM, 31:1–31:13.

[111] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17, 3 (2020), 261–272. https://doi.org/10.1038/s41592-019-0686-2

[112] Aleksandar Vitorovic, Mohammed Elseidy, and Christoph Koch. 2016. Load balancing and skew resilience for parallel joins. In *ICDE*. IEEE Computer Society, 313–324.

[113] Jeffrey Scott Vitter. 1985. Random Sampling with a Reservoir. *ACM Trans. Math. Softw.* 11, 1 (1985), 37–57.

[114] TaiNing Wang and Chee-Yong Chan. 2018. Improving Join Reorderability with Compensation Operators. In *SIGMOD Conference*. ACM, 693–708.

[115] TaiNing Wang and Chee-Yong Chan. 2020. Improved Correlated Sampling for Join Size Estimation. In *ICDE*. IEEE, 325–336.

[116] Marcel Weisgut, Daniel Ritter, Florian Schmeller, Pınar Tözün, and Tilmann Rabl. 2025. CXL-Bench: Benchmarking Shared CXL Memory Access. In *VLDB Workshops*. VLDB.org.

[117] Marcel Weisgut, Daniel Ritter, Pınar Tözün, Lawrence Benson, and Tilmann Rabl. 2025. CXL Memory Performance for In-Memory Data Processing. *Proc. VLDB Endow.* 18, 9 (2025), 3119–3133.

[118] Samuel Williams. 2008. Williams/webbase-1M. https://sparse.tamu.edu/Williams/webbase-1M Accessed: October 2026.

[119] Guoqing Xiao, Chuanghui Yin, Tao Zhou, Xueqi Li, Yuedan Chen, and Kenli Li. 2024. A Survey of Accelerating Parallel Sparse Linear Algebra. *ACM Comput. Surv.* 56, 1 (2024), 21:1–21:38.

[120] Weipeng P. Yan and Per-Åke Larson. 1994. Performing Group-By before Join. In *Proceedings of the Tenth International Conference on Data Engineering, February 14-18, 1994, Houston, Texas, USA*. IEEE Computer Society, 89–100.

[121] Weipeng P. Yan and Per-Åke Larson. 1995. Eager Aggregation and Lazy Aggregation. In *VLDB*. Morgan Kaufmann, 345–357.

[122] Dezhong Yao, Sifan Zhao, Tongtong Liu, Gang Wu, and Hai Jin. 2024. Ap-SpGEMM: Accelerating Large-scale SpGEMM with Heterogeneous Collaboration and Adaptive Panel. *ACM Transactions on Architecture and Code Optimization* (2024).

[123] Abdurrahman Yaşar, Sivasankaran Rajamanickam, Michael Wolf, Jonathan Berry, and Ümit V Çatalyürek. 2018. Fast triangle counting using cilk. In *2018 IEEE High Performance extreme Computing Conference (HPEC)*. IEEE, 1–7.

[124] Mustafa Yazici. 2023. goodbooks 10k the latest. https://www.kaggle.com/datasets/mustafayazici/goodbooks-10k-the-latest. Accessed: 2026-02-21.

[125] Serif Yesil, Azin Heidarshenas, Adam Morrison, and Josep Torrellas. 2020. Speeding up SpMV for power-law graph analytics by enhancing locality & vectorization. In *SC*. IEEE/ACM, 86.

[126] Dongxiang Zhang, Dongsheng Li, Long Guo, and Kian-Lee Tan. 2022. Unsupervised Entity Resolution With Blocking and Graph Algorithms. *IEEE Trans. Knowl. Data Eng.* 34, 3 (2022), 1501–1515.